```
                          n_estimators=500)
xg_step.fit(X_train, y_train,
       eval_set=[(X_train, y_train),
                 (X_test, y_test)
                ]
      )
```

The shap library works well in Jupyter. It provides JavaScript visualizations that allow some interactivity. To enable the JavaScript extension, we need to run the `initjs` function. Then, we create an instance of a `TreeExplainer`. This can provide the SHAP values for us. The `TreeExplainer` instance is callable and returns an `Explanation` object. This object has an attribute, `.values`, with the SHAP values for every feature for each sample.

```
import shap
shap.initjs()

shap_ex = shap.TreeExplainer(xg_step)
vals = shap_ex(X_test)
```

I'll stick the values in a DataFrame so we can see what they look like.

```
>>> shap_df = pd.DataFrame(vals.values, columns=X_test.columns)
>>> print(shap_df)
          age  education  years_exp  compensation    python         r  \
0    0.426614   0.390184  -0.246353      0.145825 -0.034680  0.379261
1    0.011164  -0.131144  -0.292135     -0.014521  0.016003 -1.043464
2   -0.218063  -0.140705  -0.411293      0.048281  0.424516  0.487451
3   -0.015227  -0.299068  -0.426323     -0.205840 -0.125867  0.320594
4   -0.468785  -0.200953  -0.230639      0.064272  0.021362  0.355619
..        ...        ...        ...           ...       ...       ...
900  0.268237  -0.112710   0.330096     -0.209942  0.012074 -1.144335
901  0.154642   0.572190  -0.227121      0.448253 -0.057847  0.290381
902  0.079129  -0.095771   1.136799      0.150705  0.133260  0.484103
903 -0.206584   0.430074  -0.385100     -0.078808 -0.083052 -0.992487
904  0.007351   0.589351   1.485712      0.056398 -0.047231  0.373149

          sql   Q1_Male  Q1_Female  Q1_Prefer not to say  \
0   -0.019017  0.004868   0.000877              0.002111
1    0.020524  0.039019   0.047712              0.001010
2   -0.098703 -0.004710   0.063545              0.000258
3   -0.062712  0.019110   0.012257              0.002184
4   -0.083344 -0.017202   0.002754              0.001432
..        ...       ...        ...                   ...
900 -0.065815  0.028274   0.032291              0.001012
901 -0.069114  0.006243   0.007443              0.002198
902 -0.120819  0.012034   0.057516              0.000266
903 -0.088811  0.080561   0.028648              0.000876
904 -0.105290  0.029283   0.074762              0.001406

     Q1_Prefer to self-describe  Q3_United States of America  Q3_India  \
0                           0.0                     0.033738 -0.117918
1                           0.0                     0.068171  0.086444
```

```
2                          0.0                    0.005533 -0.105534
3                          0.0                   -0.000044  0.042814
4                          0.0                    0.035772 -0.073206
..                         ...                         ...       ...
900                        0.0                   -0.086408  0.136677
901                        0.0                   -0.074364  0.115520
902                        0.0                    0.103810 -0.097848
903                        0.0                    0.045213  0.066553
904                        0.0                   -0.031587  0.117050

     Q3_China  major_cs  major_other  major_eng  major_stat
0   -0.018271  0.369876     0.014006  -0.013465    0.104177
1   -0.026271 -0.428484    -0.064157  -0.026041    0.069931
2   -0.010548 -0.333695     0.016919  -0.026932   -0.591922
3   -0.024099  0.486864     0.038438  -0.013727    0.047564
4   -0.022188  0.324419     0.012664  -0.019550    0.093926
..        ...       ...          ...        ...         ...
900  0.310404 -0.407444    -0.013195  -0.026412   -0.484734
901 -0.008244  0.602087     0.039680  -0.012820    0.083934
902  0.003234 -0.313785    -0.080046  -0.066032    0.101975
903 -0.031448 -0.524141    -0.048108  -0.007185    0.093196
904  0.008734 -0.505613    -0.159411  -0.067388    0.126560

[905 rows x 18 columns]
          age  education  years_exp  compensation    python         r  \
0    0.426614   0.390184  -0.246353      0.145825 -0.034680  0.379261
1    0.011164  -0.131144  -0.292135     -0.014521  0.016003 -1.043464
2   -0.218063  -0.140705  -0.411293      0.048281  0.424516  0.487451
3   -0.015227  -0.299068  -0.426323     -0.205840 -0.125867  0.320594
4   -0.468785  -0.200953  -0.230639      0.064272  0.021362  0.355619
..        ...        ...        ...           ...       ...       ...
900  0.268237  -0.112710   0.330096     -0.209942  0.012074 -1.144335
901  0.154642   0.572190  -0.227121      0.448253 -0.057847  0.290381
902  0.079129  -0.095771   1.136799      0.150705  0.133260  0.484103
903 -0.206584   0.430074  -0.385100     -0.078808 -0.083052 -0.992487
904  0.007351   0.589351   1.485712      0.056398 -0.047231  0.373149

          sql   Q1_Male  Q1_Female  Q1_Prefer not to say  \
0   -0.019017  0.004868   0.000877              0.002111
1    0.020524  0.039019   0.047712              0.001010
2   -0.098703 -0.004710   0.063545              0.000258
3   -0.062712  0.019110   0.012257              0.002184
4   -0.083344 -0.017202   0.002754              0.001432
..        ...       ...        ...                   ...
900 -0.065815  0.028274   0.032291              0.001012
901 -0.069114  0.006243   0.007443              0.002198
902 -0.120819  0.012034   0.057516              0.000266
903 -0.088811  0.080561   0.028648              0.000876
904 -0.105290  0.029283   0.074762              0.001406

     Q1_Prefer to self-describe  Q3_United States of America  Q3_India  \
0                           0.0                    0.033738 -0.117918
```

```
1                         0.0                    0.068171  0.086444
2                         0.0                    0.005533 -0.105534
3                         0.0                   -0.000044  0.042814
4                         0.0                    0.035772 -0.073206
..                        ...                         ...       ...
900                       0.0                   -0.086408  0.136677
901                       0.0                   -0.074364  0.115520
902                       0.0                    0.103810 -0.097848
903                       0.0                    0.045213  0.066553
904                       0.0                   -0.031587  0.117050

     Q3_China  major_cs  major_other  major_eng  major_stat
0   -0.018271  0.369876     0.014006  -0.013465    0.104177
1   -0.026271 -0.428484    -0.064157  -0.026041    0.069931
2   -0.010548 -0.333695     0.016919  -0.026932   -0.591922
3   -0.024099  0.486864     0.038438  -0.013727    0.047564
4   -0.022188  0.324419     0.012664  -0.019550    0.093926
..        ...       ...          ...        ...         ...
900  0.310404 -0.407444    -0.013195  -0.026412   -0.484734
901 -0.008244  0.602087     0.039680  -0.012820    0.083934
902  0.003234 -0.313785    -0.080046  -0.066032    0.101975
903 -0.031448 -0.524141    -0.048108  -0.007185    0.093196
904  0.008734 -0.505613    -0.159411  -0.067388    0.126560

[905 rows x 18 columns]
```

If you add up each row and add the `.base_values` attribute (this is the default guess for the model) you will get the log odds value that the sample is in the positive case. I will stick the sum in a dataframe with the column name *pred*. I will also include the ground truth value and convert the log odds sum into a probability column, *prob*. If the probability is above .5 (this happens when *pred* is positive), we predict the positive value.

```
>>> print(pd.concat([shap_df.sum(axis='columns')
...                       .rename('pred') + vals.base_values,
...    pd.Series(y_test, name='true')], axis='columns')
...    .assign(prob=lambda adf: (np.exp(adf.pred) /
...                              (1 + np.exp(adf.pred))))
... )
        pred  true      prob
0   1.204692     1  0.769358
1  -2.493559     0  0.076311
2  -2.205473     0  0.099260
3  -0.843847     1  0.300725
4  -0.168726     1  0.457918
..       ...   ...       ...
900 -1.698727     0  0.154632
901  1.957872     0  0.876302
902  0.786588     0  0.687098
903 -2.299702     0  0.091148
904  1.497035     1  0.817132
```

```
[905 rows x 3 columns]
```

Note that for index rows 3, 4, 901, and 902, our model makes incorrect predictions.

19.2 Examining a Single Prediction

If we want to explore how `shap` attributes the prediction from the features, you can create a few visualizations.

Let's explore the first entry in the test data. It looks like this:

```
>>> X_test.iloc[0]
age                            22.0
education                      16.0
years_exp                       1.0
compensation                    0.0
python                          1.0
r                               0.0
sql                             0.0
Q1_Male                         1.0
Q1_Female                       0.0
Q1_Prefer not to say            0.0
Q1_Prefer to self-describe      0.0
Q3_United States of America     0.0
Q3_India                        1.0
Q3_China                        0.0
major_cs                        1.0
major_other                     0.0
major_eng                       0.0
major_stat                      0.0
Name: 7894, dtype: float64
```

Our model predicts a value of 1 (which is also the ground truth), which corresponds to a Software Engineer. Is that because of *age*, *education*, *R*, or something else?

```
>>> # predicts software engineer... why?
>>> xg_step.predict(X_test.iloc[[0]])
array([1])

>>> # ground truth
>>> y_test[0]
1
```

We can also do some math to validate the SHAP values. We start off with the expected value.

```
>>> # Since this is below zero, the default is Data Scientist
>>> shap_ex.expected_value
-0.2166416
```

Then we add the sum of the values from the row to the expected value.

```
>>> # > 0 therefore ... Software Engineer
>>> shap_ex.expected_value + vals.values[0].sum()
1.2046916
```

Because this value is above 0, we would predict Software Engineer for this sample.

19.3 Waterfall Plots

Let's make a *waterfall* plot to explore the SHAP values. This plots an explanation of a single prediction. It displays how the SHAP value from each column impacts the result. You can see a vertical line at -0.21. This represents the default or base value. This is the right edge of the bottom bar. By default, our model would predict Data Scientist. But we need to look at each of the features and examine how they push the prediction.

The waterfall plot is rank ordered. The feature with the most impact is shown at the top. The next most important feature is second, and so on. The *age* value of 22 gave a SHAP value of 0.43. This pushes the result towards the positive case. The *R* value, 0.0, also pushes toward the positive case with a magnitude of 0.38. We repeat this and see that the values add up to 1.204 (in the upper right). (You can confirm this from our addition above.)

```
fig = plt.figure(figsize=(8, 4))
shap.plots.waterfall(vals[0], show=False)
```

To get a feel for how the values of this sample relate to the other samples, we can plot a histogram showing the distributions of the values and mark our value.

I will write a function to do this, `plot_histograms`.

```
def plot_histograms(df, columns, row=None, title='', color='shap'):
    """
    Parameters
    ----------
    df : pandas.DataFrame
        The DataFrame to plot histograms for.
    columns : list of str
        The names of the columns to plot histograms for.
    row : pandas.Series, optional
        A row of data to plot a vertical line for.
    title : str, optional
        The title to use for the figure.
    color : str, optional
        'shap' - color positive values red. Negative blue
        'mean' - above mean red. Below blue.
        None - black

    Returns
    -------
    matplotlib.figure.Figure
        The figure object containing the histogram plots.
    """
    red = '#ff0051'
    blue = '#008bfb'
```

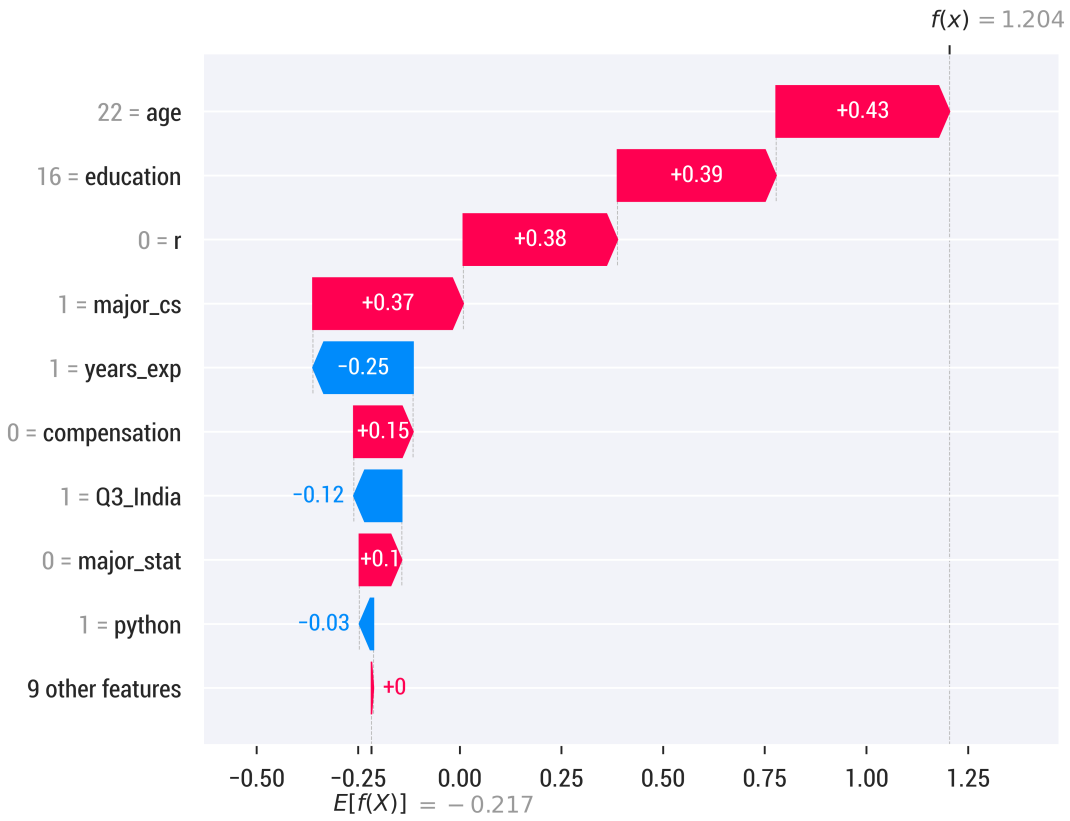


Figure 19.1: SHAP waterfall plot of first test sample

```
fig, ax = plt.subplots(figsize=(8, 4))
hist = (df
 [columns]
 .hist(ax=ax, color='#bbb')
)
fig = hist[0][0].get_figure()
if row is not None:
    name2ax = {ax.get_title():ax for ax in fig.axes}
    pos, neg = red, blue
    if color is None:
        pos, neg = 'black', 'black'
    for column in columns:
        if color == 'mean':
            mid = df[column].mean()
        else:
            mid = 0
        if row[column] > mid:
            c = pos
        else:
            c = neg
        name2ax[column].axvline(row[column], c=c)
```

```
    fig.tight_layout()
    fig.suptitle(title)
    return fig
```

I will use this function to show where the features of the individual sample are located among the distributions.

```
features = ['education', 'r', 'major_cs', 'age', 'years_exp',
            'compensation']
fig = plot_histograms(shap_df, features, shap_df.iloc[0],
                      title='SHAP values for row 0')
```

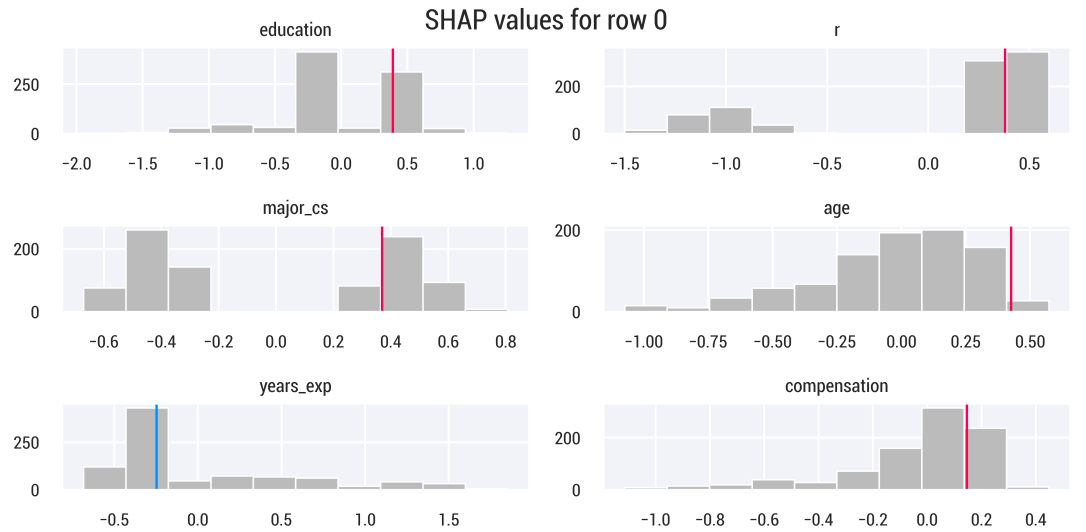


Figure 19.2: Histograms of SHAP values for first row

We can also use this function to visualize the histograms of the original feature values.

```
fig = plot_histograms(X_test, features, X_test.iloc[0],
                      title='Values for row 0', color='mean')
```

We can create a bar plot of the SHAP values with the Pandas library.

```
fig, ax = plt.subplots(figsize=(8, 4))
(pd.Series(vals.values[0], index=X_test.columns)
 .sort_values(key=np.abs)
 .plot.barh(ax=ax)
)
```

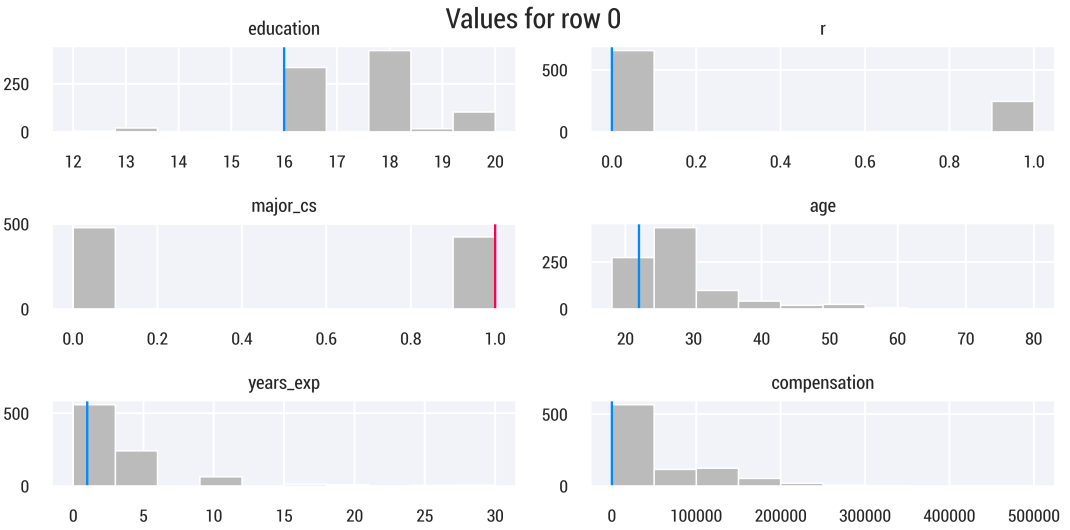


Figure 19.3: Histograms of original values for first row

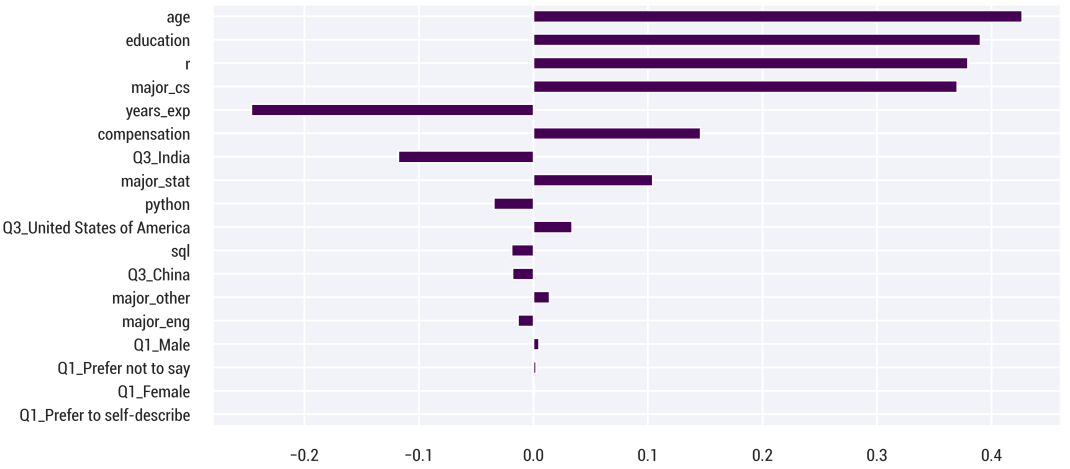


Figure 19.4: Bar plot of SHAP values for first test sample

19.4 A Force Plot

The shap library also provides a flattened version of the waterfall plot, called a *force plot*. (In my example, I show `matplotlib=True`, but in Jupyter, you can leave that out and the plot will have some interactivity.)

```
# use matplotlib if having js issues
# blue - DS
# red - Software Engineer
# to save need both matplotlib=True, show=False
res = shap.plots.force(base_value=vals.base_values,
                       shap_values=vals.values[0,:], features=X_test.iloc[0],
                       matplotlib=True, show=False
)
res.savefig('img/shap_forceplot0.png', dpi=600, bbox_inches='tight')
```

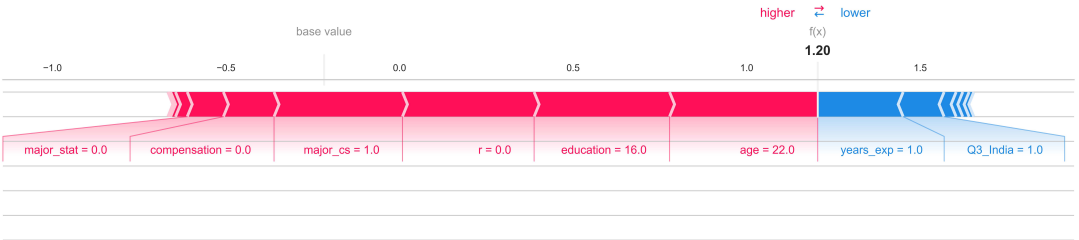


Figure 19.5: Force plot of SHAP values for first test sample

19.5 Force Plot with Multiple Predictions

The shap library allows you to pass in multiple rows of SHAP values into the `force` function. In that case, it flips them vertically and stacks them next to each other. There currently is no Matplotlib version for this plot, and the Jupyter / JavaScript version has a dropdown to change the order of the results and a dropdown to change what features it shows.

```
# First n values
n = 100
# blue - DS
# red - Software Engineer
shap.plots.force(base_value=vals.base_values,
                 shap_values=vals.values[:n,:], features=X_test.iloc[:n],
                 )
```

19.6 Understanding Features with Dependence Plots

Because the shap library has computed SHAP values for every feature, we can use that to visualize how features impact the model. The *Dependence Scatter Plot* shows the SHAP values across the values for a feature. This plot can be a little confusing at first, but when you take a moment to understand it, it becomes very useful.

Let's pick one of the features that had a big impact on our model, *education*.

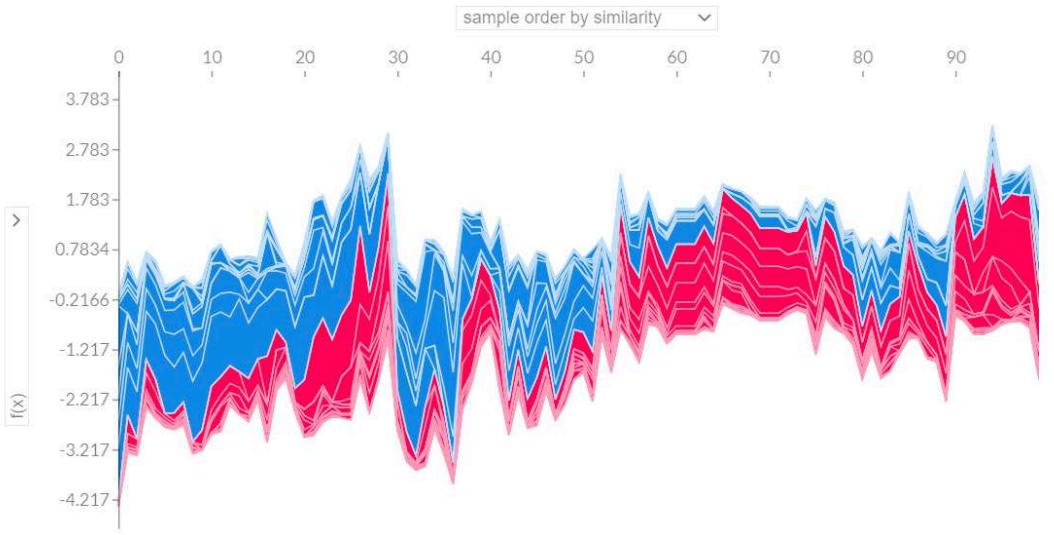


Figure 19.6: Force plot of SHAP values for first 100 test samples

```
fig, ax = plt.subplots(figsize=(8, 4))
shap.plots.scatter(vals[:, 'education'], ax=ax, color=vals,
                   x_jitter=0, hist=False)
```

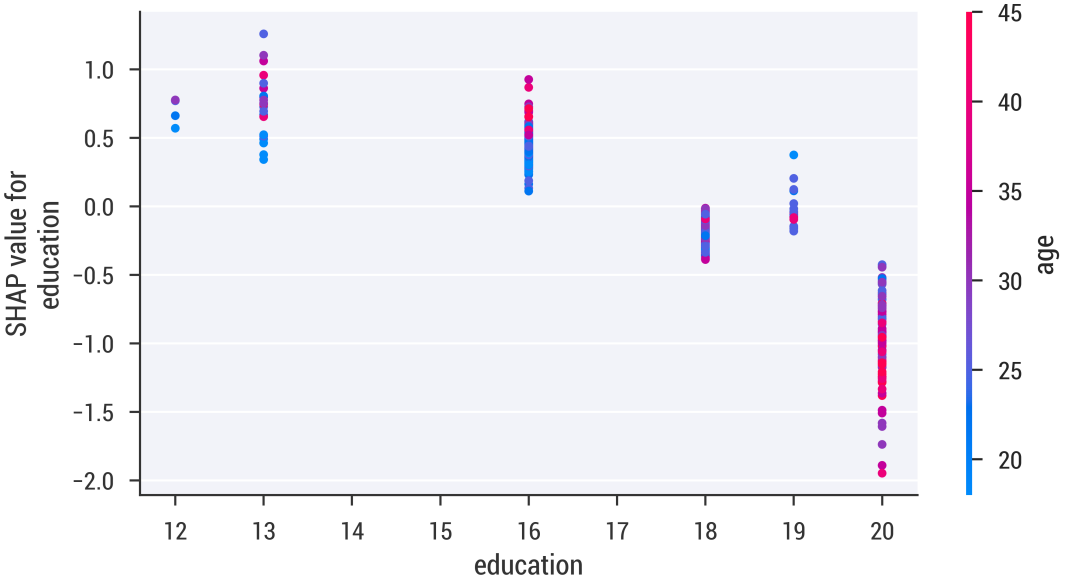


Figure 19.7: Dependence plot of SHAP values for the education feature

Let's try and make sense of this plot. In the x-axis, we have different entries for *education*. In the y-axis, we have the different SHAP values for each education value. You might wonder why the y values are different. This is because the *education* column interacts with other columns, so sometimes a value of 19 pushes toward Data Scientist, and sometimes it moves towards Software Engineer. Remember that y values above 0 will push toward the positive label, and negative values push toward the negative label. My interpretation of this plot is that there is a non-linear relationship between education and the target. Education values push towards the negative label quicker as the value increases.

Here are a couple of other things to note about this plot. The `shap` library automatically chooses an *interaction index*, another column to plot in a color gradient. In this example, it decided on the *age* feature. This allows you to visualize how another column interacts with *education*. It is a little hard to see, but higher compensation pushes more toward the software engineer label for low education levels. You can specify a different column by setting the `color` parameter to a single column of values (`color=vals[:, 'compensation']`)).

19.7 Jittering a Dependence Plot

Also, because the *education* entries are limited to a few values, the scatter plot just plots the values on top of each other. This makes it hard to understand what is really happening. We will address that using the `x_jitter` parameter to spread the values. I also like to adjust the `alpha` parameter to adjust the transparency of each dot. By default, every dot has a transparency of 1, which means that it is completely opaque. If you lower the value, it makes them more see-through and lets you see the density better.

Here is a plot that sets the alpha to .5, spreads out the values in the x-axis with `x_jitter`, and shows the interaction with the *years_exp* column.

```
fig, ax = plt.subplots(figsize=(8, 4))
shap.plots.scatter(vals[:, 'education'], ax=ax, color=vals[:, 'years_exp'], x_jitter=1,
                   alpha=.5)
```

This plot makes it very clear that most of the *education* entries have values of 16 and 18. Based on the SHAP values, having more schooling pushes toward data science.

You can also see a faint histogram. Because most values have an education level above 15, you might want to explore whether the model is overfitting the values for 12 and 13. The value for 19 might warrant exploration as there are few entries there.

The jittering and histogram functionality is automatic when you use the `scatter` function.

Let's explore one more dependence plot showing the impact of the *major_cs* column. We will choose the *R* column for the interaction index.

```
fig, ax = plt.subplots(figsize=(8, 4))
shap.plots.scatter(vals[:, 'major_cs'], ax=ax, color=vals[:, 'r'], alpha=.5)
```

This plot indicates that studying computer science pushes the result more toward software engineer than the reverse does toward data scientist. It is interesting to note that adding R to studying CS strengthens the effect (which is the opposite of what I would have expected).

19.8 Heatmaps and Correlations

Though not part of the `shap` package, it can be helpful to look at the heatmaps of the correlations of the SHAP values.

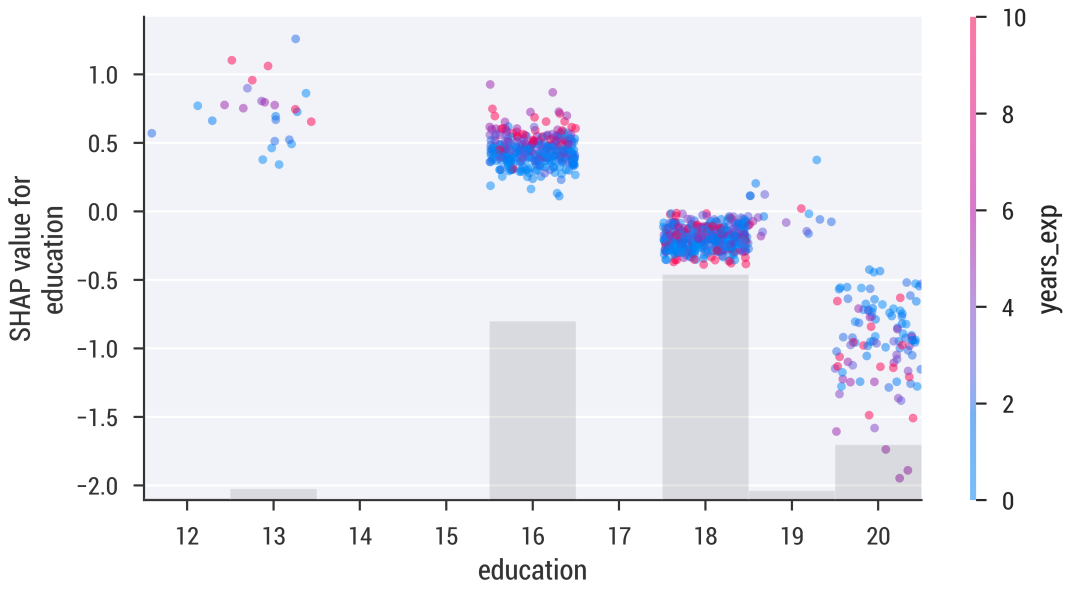


Figure 19.8: Dependence plot of SHAP values for education feature with jittering and different interaction column

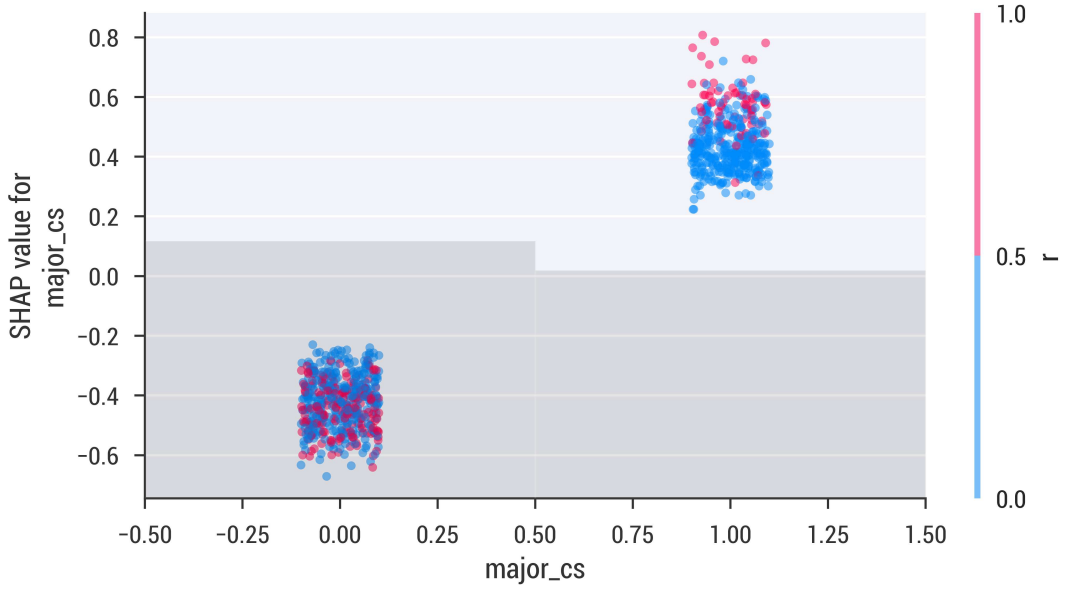


Figure 19.9: Dependence plot of SHAP values for education feature with jittering for major_cs column

Let's start with the standard correlation of the features (not involving SHAP).

```
import seaborn as sns
fig, ax = plt.subplots(figsize=(8, 4))
sns.heatmap(X_test
            .assign(software_eng=y_test)
            .corr(method='spearman')
            .loc[:, ['age', 'education', 'years_exp',
                     'compensation', 'r', 'major_cs',
                     'software_eng']],
            cmap='RdBu', annot=True, fmt='.2f', vmin=-1, vmax=1, ax=ax
)
```

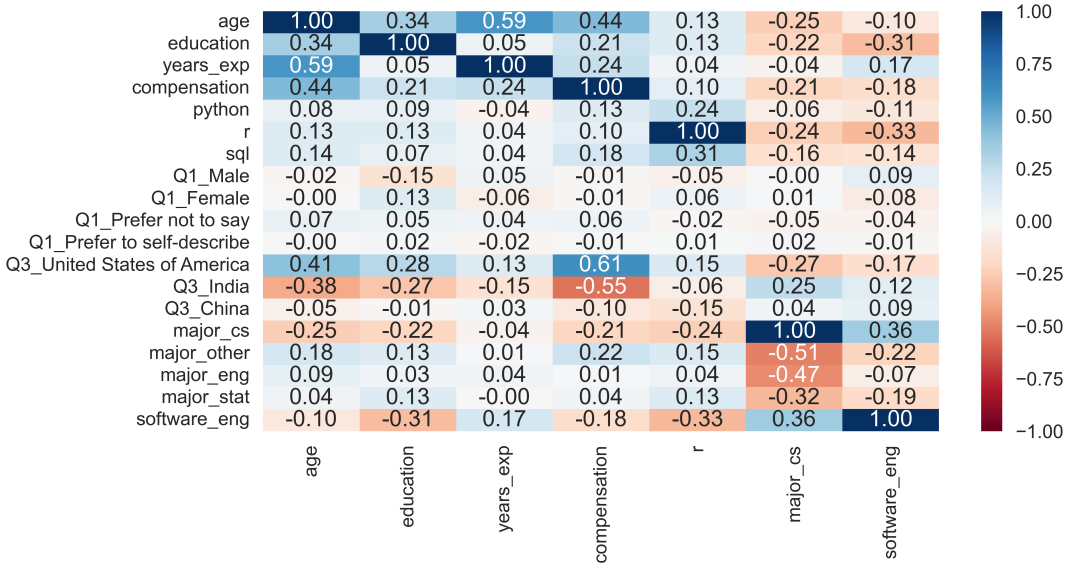


Figure 19.10: Correlation heatmap between subset of features for the testing data

This heatmap tells us if the features tend to move in the same or opposite directions. I have added the *software_eng* column to see if there is a correlation with the prediction.

Now, we will create a heatmap of *the correlation of the SHAP values for each prediction*. This correlation will tell us if two features tend to move the prediction similarly. For example, the SHAP values for both *education* and *major_stat* push the prediction label in the same direction. This can help us understand interactions from different points of view.

```
import seaborn as sns
fig, ax = plt.subplots(figsize=(8, 4))
sns.heatmap(shap_df
            .assign(software_eng=y_test)
            .corr(method='spearman')
            .loc[:, ['age', 'education', 'years_exp',  'compensation', 'r', 'major_cs',
                     'software_eng']],
            cmap='RdBu', annot=True, fmt='.2f', vmin=-1, vmax=1, ax=ax
```

)

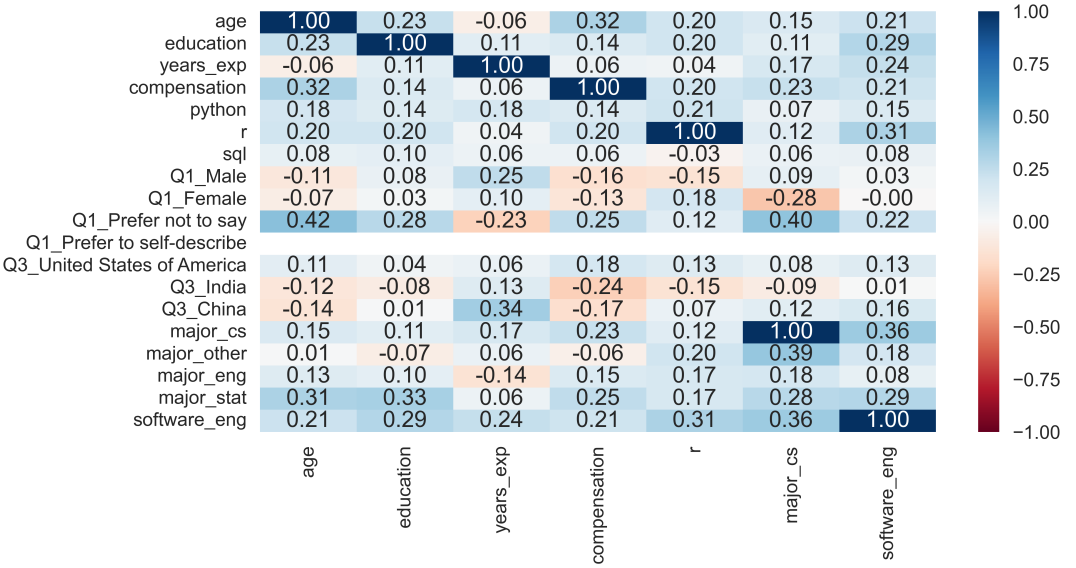


Figure 19.11: Correlation heatmap between subset of SHAP values for each feature for the testing data

Generally, when dealing with correlation heatmaps, I want to look at the dark red and dark blue values (ignoring the self-correlation values). It looks like the SHAP values for *compensation* tend to rise if the SHAP values for *age* rise. (Note that this is different from the actual non-SHAP values correlating, though, in the case of *compensation* and *age*, it looks like they do.) You could explore these further by doing a scatter plot.

19.9 Beeswarm Plots of Global Behavior

One of the reasons that SHAP is so popular is that it not only explains local predictions and feature interactions but also gives you a global view of how features impact the model.

Let's look at the beeswarm or summary plot. This provides a rank-ordered listing of the features that drive the most impact on the final prediction. In the x-axis is the SHAP value. Positive value push towards the positive label. Each feature is colored to indicate high (red) or low (blue) values.

The *R* feature is binary, with only red and blue values. You can see that a high value (1) results in a significant push toward data science. The spread on the red values indicates some interaction with other columns. Another way to understand this is that there are probably some low frequency effects when *R* is red that cause a large impact on our model.

The *years_exp* feature has gradations in the color because there are multiple options for that feature. It looks like there is a pretty smooth transition from blue to red. Contrast that smooth gradation with *education*, where you see from the left red, then purple, then red again, then blue. This indicates non-monotonic behavior for the *education* feature. We saw this in the dependence plot for *education* above.

```
fig = plt.figure(figsize=(8, 4))
shap.plots.beeswarm(vals)
```

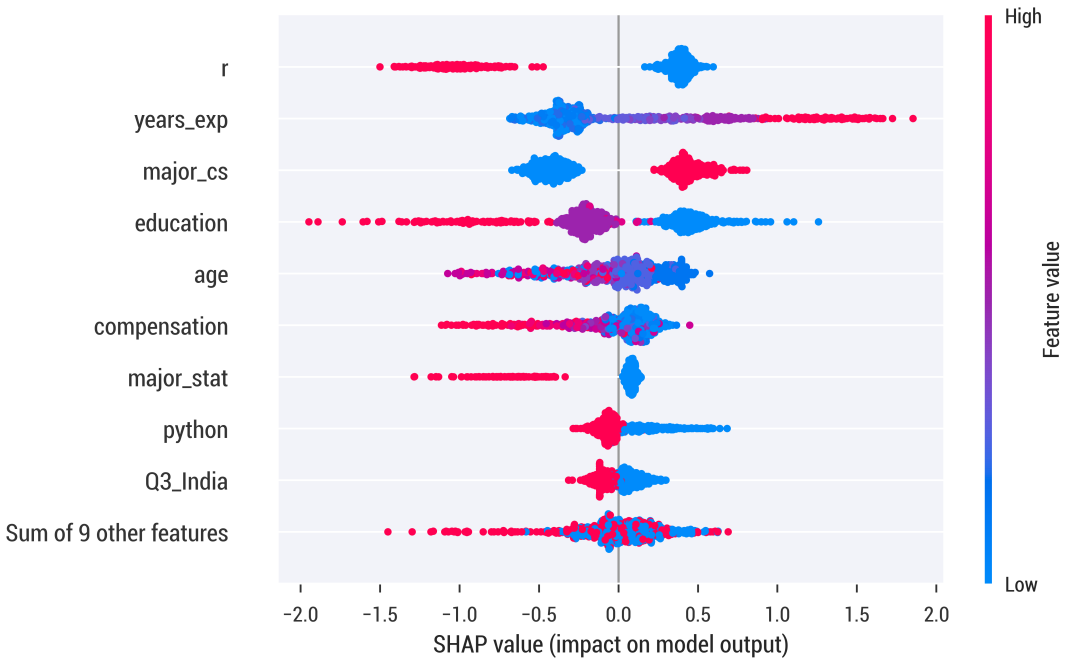


Figure 19.12: Summary plot of SHAP values for top features

If you want to see all of the features, use the `max_display` parameter. I'm also changing the colormap to a Matplotlib colormap that will work better in grayscale. (This is for all those who bought the grayscale physical edition.)

```
from matplotlib import cm
fig = plt.figure(figsize=(8, 4))
shap.plots.beeswarm(vals, max_display=len(X_test.columns), color=cm.autumn_r)
```

19.10 SHAP with No Interaction

If your model has interactions, SHAP values will reflect them. If we remove interactions, you will simplify your model (by making the `max_depth` equal to 1). It will also make non-linear responses more clear.

Let's train a model of stumps and look at some of the SHAP plots.

```
no_int_params = {'random_state': 42,
                 'max_depth': 1
}
xg_no_int = xgb.XGBClassifier(**no_int_params, early_stopping_rounds=50,
                              n_estimators=500)
```

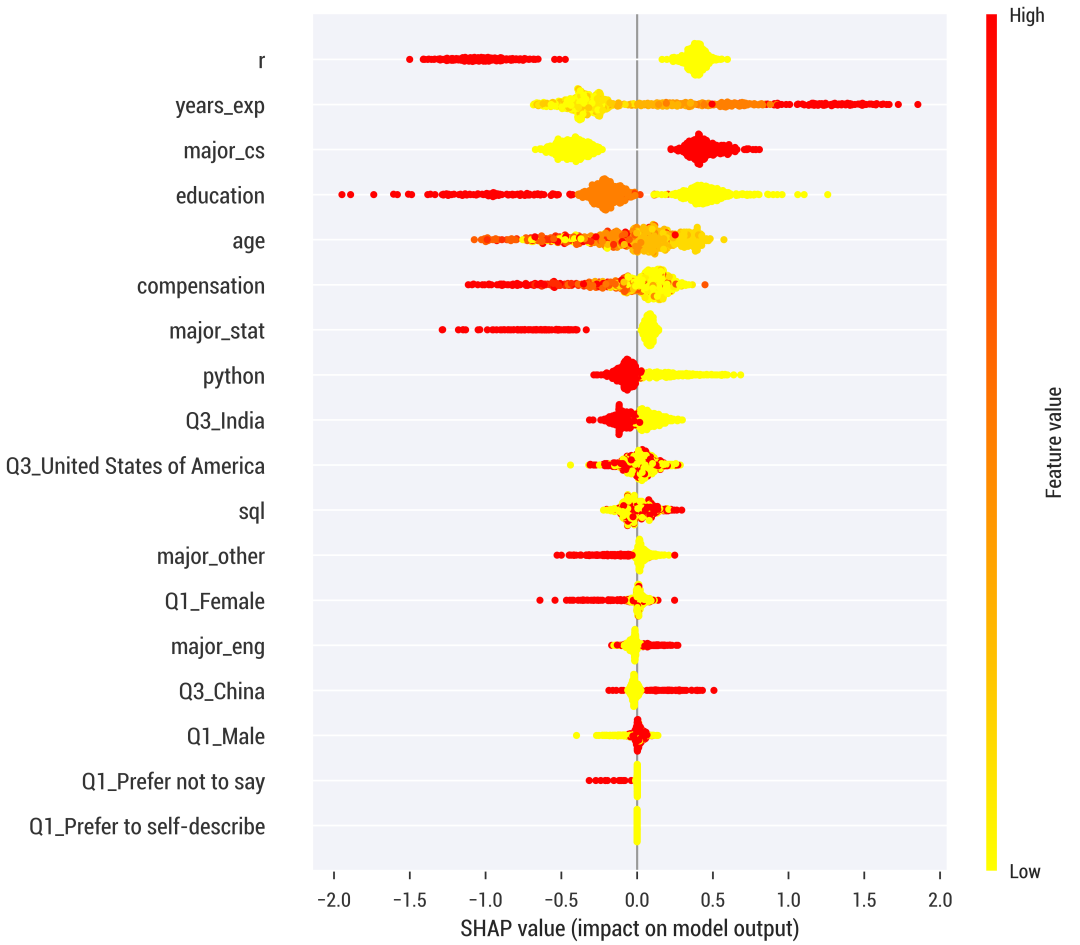


Figure 19.13: Summary plot of SHAP values for all the features

```
xg_no_int.fit(X_train, y_train,
       eval_set=[(X_train, y_train),
                 (X_test, y_test)
                ]
)
```

The stump model does not per form as well as the default model. But it is close. The default score was 0.74.

```
>>> xg_no_int.score(X_test, y_test)
0.7370165745856354

shap_ind = shap.TreeExplainer(xg_no_int)
shap_ind_vals = shap_ind(X_test)
```

Here is the summary plot for the model without interactions.

```
from matplotlib import cm
fig = plt.figure(figsize=(8, 4))
shap.plots.beeswarm(shap_ind_vals, max_display=len(X_test.columns))
```

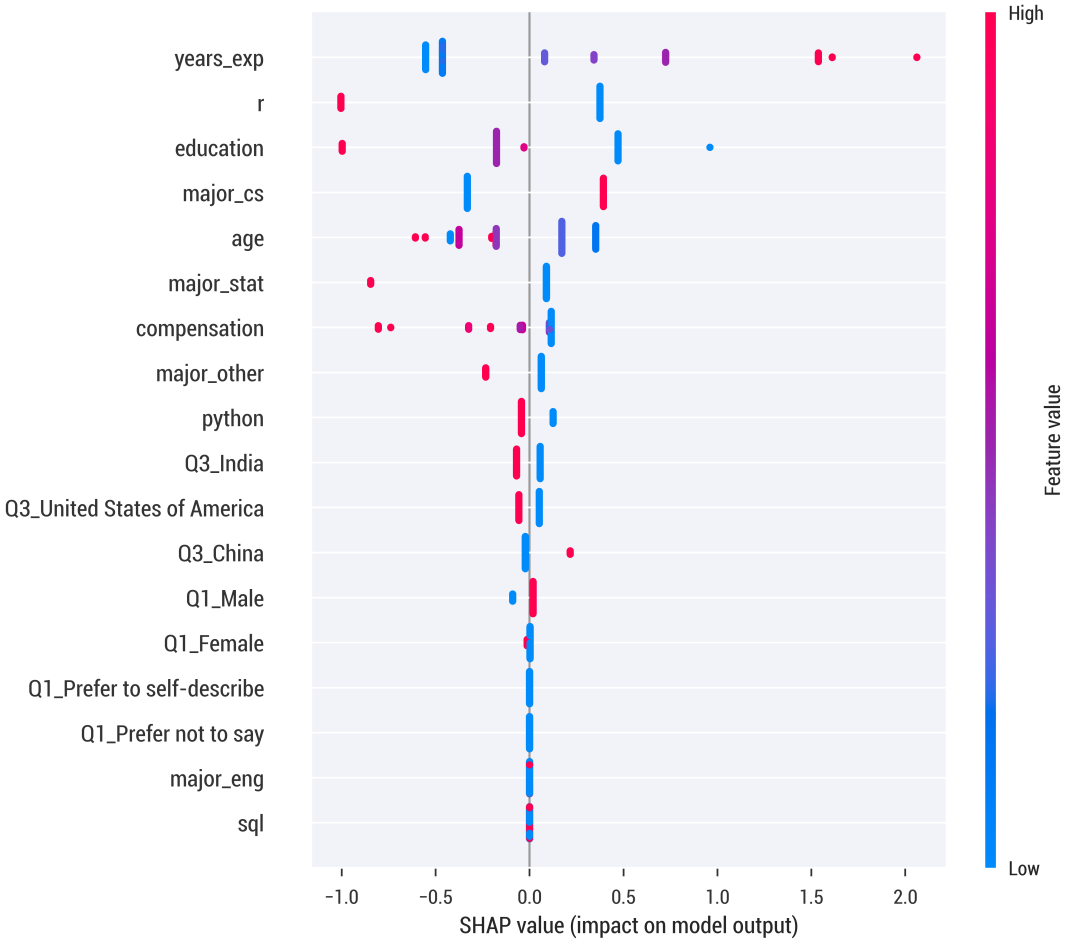


Figure 19.14: Summary plot of SHAP values for all the features without interactions

It is interesting to observe that the ordering of feature importance changes with this model.

Here is the *years_exp* plot for our model with interactions. You can see a vertical spread in the y-axis from the interactions of *years_exp* with other columns. (The spread in the x-axis is due to jittering.)

```
fig, ax = plt.subplots(figsize=(8, 4))
shap.plots.scatter(vals[:, 'years_exp'], ax=ax,
                   color=vals[:, 'age'], alpha=.5,
                   x_jitter=1)
```

Here is the dependence plot for the model without interactions. You can see that there is no variation in the y-axis.

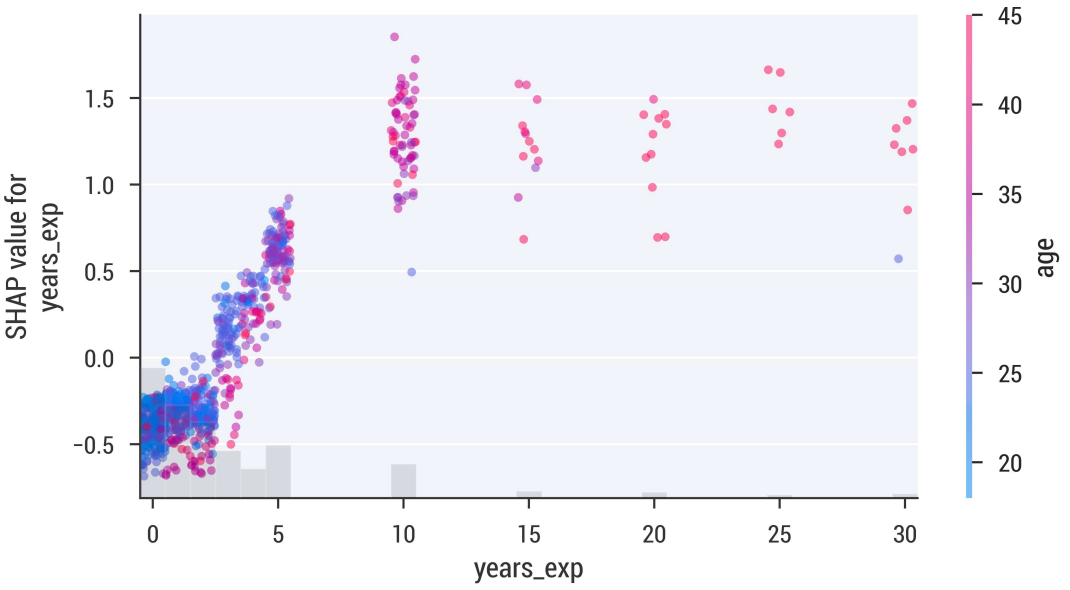


Figure 19.15: Dependence plot of SHAP values for years_exp feature with feature interactions and jittering.

```
fig, ax = plt.subplots(figsize=(8, 4))
shap.plots.scatter(shap_ind_vals[:, 'years_exp'], ax=ax,
                   color=shap_ind_vals[:, 'age'], alpha=.5,
                   x_jitter=1)
```

This makes the non-linear response in *years_exp* very clear. Note that you might want to jitter this is the y-axis to help reveal population density, but `shap` doesn't support that. You can change the `alpha` parameter to help with density.

19.11 Summary

SHAP (SHapley Additive exPlanations) is a powerful tool for explaining the behavior of machine learning models, including XGBoost models. It provides insight into which features are most important in making predictions and how each feature contributes to the model's output. This can be useful for several reasons, including improving the interpretability of a model, identifying potential issues or biases in the data, and gaining a better understanding of how the model makes predictions.

In this chapter, we explored using SHAP values to understand predictions and interactions between the features. We also used SHAP values to rank features.

SHAP values are an important mechanism for explaining black-box models like XGBoost.

19.12 Exercises

1. How can SHAP be used to explain a model's global and local behavior?
2. How does the waterfall plot display the impact of each feature on the prediction result?
3. How does the force plot display the impact of features on the model

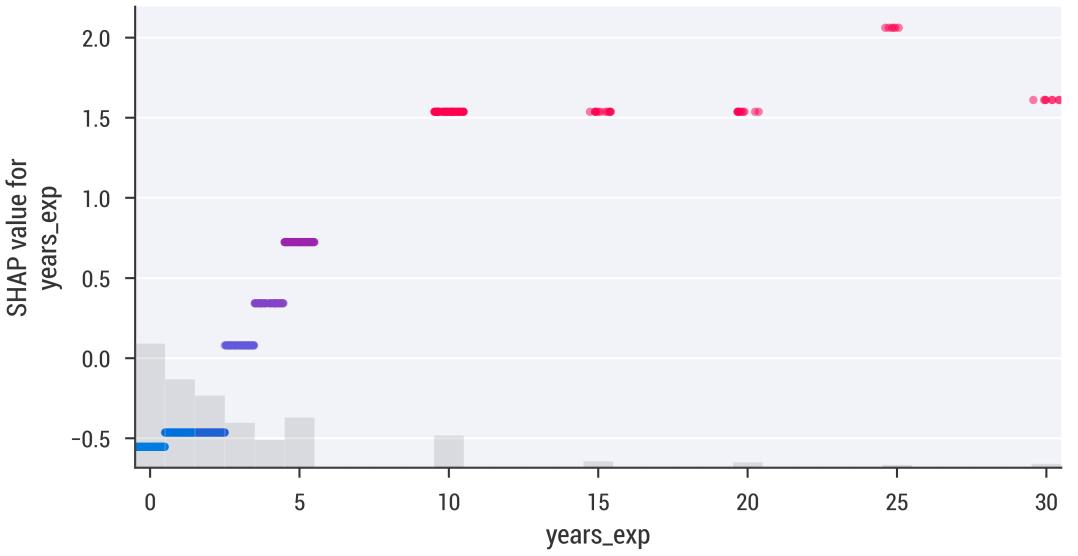


Figure 19.16: Dependence plot of SHAP values for years_exp feature with no feature interaction and jittering for major_cs column

4. What does the summary plot display, and how does it help interpret feature importance?
5. How does the dependence plot help to identify non-monotonic behavior?
6. How can the summary plot help identify features with non-monotonic behavior?
7. How can the summary plot help to identify features that have interacted with other features?

Chapter 20

Better Models with ICE, Partial Dependence, Monotonic Constraints, and Calibration

In this chapter, we will explore some advanced techniques that can be used to improve the interpretability and performance of XGBoost models. We will discuss Individual Conditional Expectation (ICE) plots and Partial Dependence Plots (PDP). These powerful visualization tools allow us to understand how the input features affect the predictions made by the model. We will also examine how to constrain XGBoost models to prevent overfitting and improve generalization performance. This chapter will provide a comprehensive overview of some important techniques that can be used to extract valuable insights from XGBoost models.

20.1 ICE Plots

Individual Conditional Expectation (ICE) plots are a useful tool for visualizing the effect of a single input variable on the output of a machine learning model. In the context of XGBoost, ICE plots can help us understand how each feature contributes to the final predictions made by the model. This section will explore using Python to create ICE plots for XGBoost models.

An ICE plot shows the predicted values of a machine learning model for a single observation as the value of a specific input feature varies. In other words, an ICE plot displays the model's output for a fixed instance while incrementally changing one input feature's value. Each line in the plot represents the predicted output for a particular instance as the input feature changes. By examining the shape and slope of each line, we can gain insights into how the model uses that input feature to make predictions. ICE plots provide a detailed view of the relationship between a single input feature and the model's predictions.

To create an ICE plot for a single row, follow these steps:

1. Choose an input feature to analyze and select a range of values for that feature to vary.
2. Fix the values of all other input features to the observed values for the instance of interest.
3. Vary the selected input feature across the values chosen in step 1.
4. For each value of the selected input feature, calculate the corresponding prediction of the model for the specific instance of interest.
5. Plot the values obtained in step 4 against the corresponding values of the input feature to create a single line in the ICE plot.

You can repeat this process for each row in your data and visualize how changing that feature would impact the final result.

```
xgb_def = xgb.XGBClassifier(random_state=42)
xgb_def.fit(X_train, y_train)
xgb_def.score(X_test, y_test)
```

Let's make the default model for the *r* and *education* features. We will use scikit-learn to create an ICE plot.

```
from sklearn.inspection import PartialDependenceDisplay
fig, axes = plt.subplots(ncols=2, figsize=(8,4))
PartialDependenceDisplay.from_estimator(xgb_def, X_train, features=['r', 'education'],
                                        kind='individual', ax=axes)
```

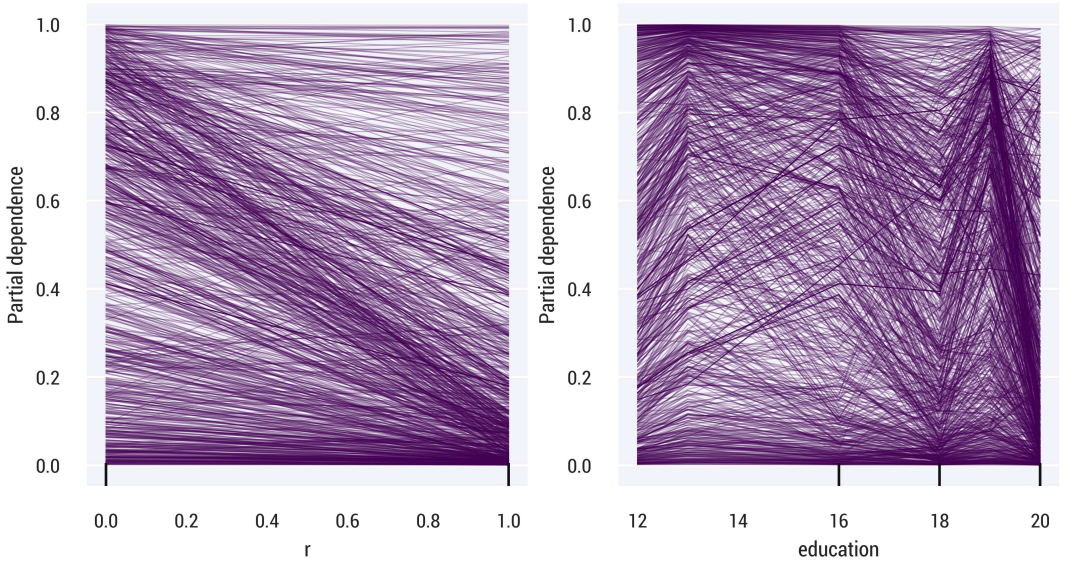


Figure 20.1: ICE plots for R and education. As R values have a positive value, the probability of data scientists tends to increase. For education, it appears that more education also tends to push toward data scientists.

It is a little difficult to discern what is happening here. Remember that the y-axis in these plots represents the probability of the final label, Software Engineer (1) or Data Scientist (0). If *r* goes to 1, there is a strong push to Data Scientist. For the *education* plot, low education values tend toward software engineering, while larger values push toward data science.

One technique to help make the visualizations more clear is to have them all start at the same value on the left-hand side. We can do that with the `centered` parameter.

```
fig, axes = plt.subplots(ncols=2, figsize=(8,4))
PartialDependenceDisplay.from_estimator(xgb_def, X_train, features=['r', 'education'],
                                        centered=True,
                                        kind='individual', ax=axes)
```

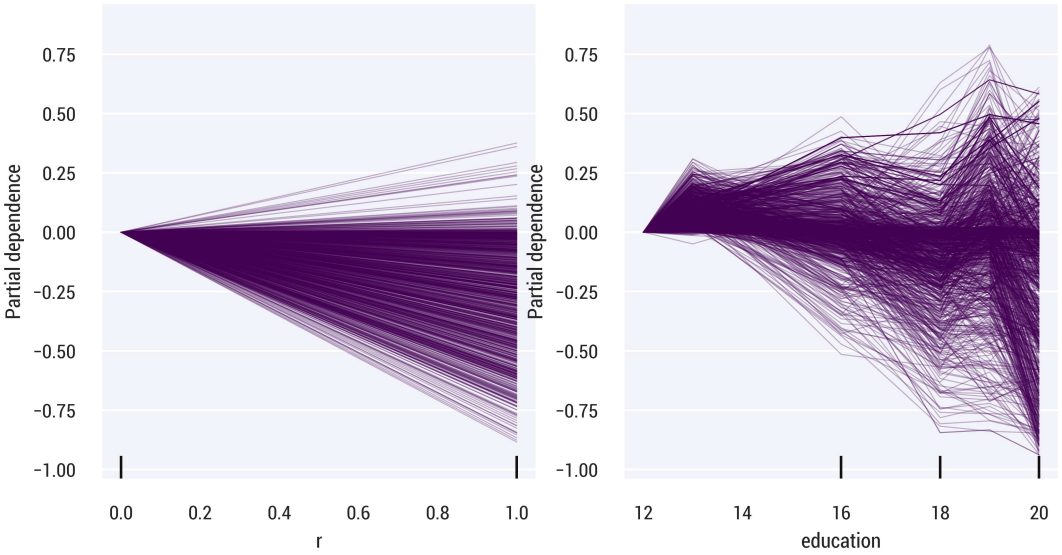


Figure 20.2: Centered ICE plots helps visualize the impact of the feature as it changes.

This plot also reveals that ticks at the bottom are intended to help visualize the number of rows with those values. However, due to the binned values in the survey data, the education levels are not discernable.

We can throw a histogram on top to see the distribution. This should provide intuition into the density of data. Locations with more data would tend to have better predictions. In the case of education, we don't have many examples of respondents with less than 14 years of education. That would cause us to have more uncertainty about those predictions.

```
fig, axes = plt.subplots(ncols=2, figsize=(8,4))
ax_h0 = axes[0].twinx()
ax_h0.hist(X_train.r, zorder=0)

ax_h1 = axes[1].twinx()
ax_h1.hist(X_train.education, zorder=0)

PartialDependenceDisplay.from_estimator(xgb_def, X_train, features=['r', 'education'],
                                        centered=True,
                                        ice_lines_kw={'zorder':10},
                                        kind='individual', ax=axes)
fig.tight_layout()
```

I wrote `quantile_ice`, a function that will subdivide a feature by the quantile of the predicted probability and create lines from the average of each quantile. It can also show a histogram.

```
def quantile_ice(clf, X, col, center=True, q=10, color='k', alpha=.5, legend=True,
                 add_hist=False, title='', val_limit=10, ax=None):
  """
```

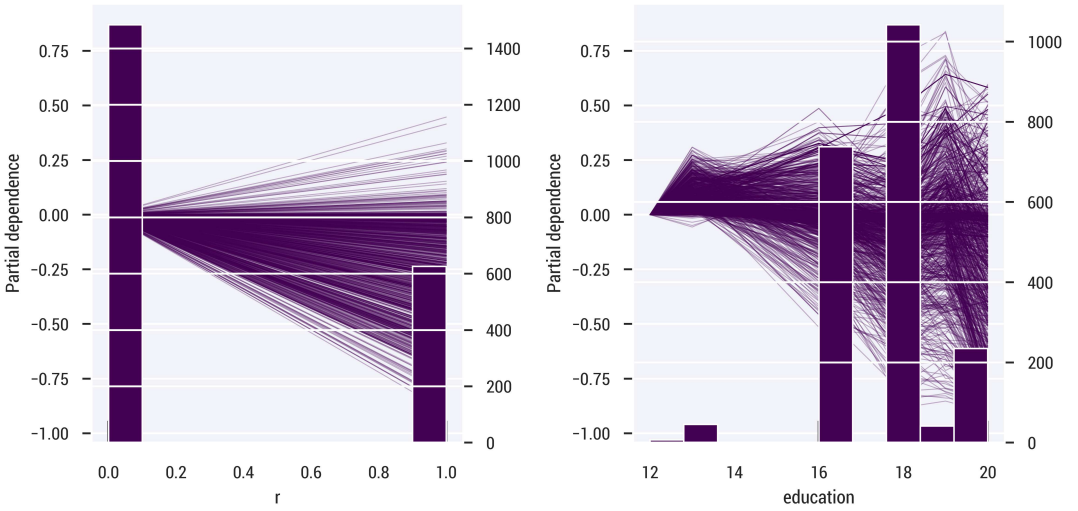


Figure 20.3: ICE plots with histograms to aid with understanding where there is sparseness in the data that might lead to overfitting.

```
Generate an ICE plot for a binary classifier's predicted probabilities split
by quantiles.

Parameters:
----------
clf : binary classifier
    A binary classifier with a `predict_proba` method.
X : DataFrame
    Feature matrix to predict on with shape (n_samples, n_features).
col : str
    Name of column in `X` to plot against the quantiles of predicted probabilities.
center : bool, default=True
    Whether to center the plot on 0.5.
q : int, default=10
    Number of quantiles to split the predicted probabilities into.
color : str or array-like, default='k'
    Color(s) of the lines in the plot.
alpha : float, default=0.5
    Opacity of the lines in the plot.
legend : bool, default=True
    Whether to show the plot legend.
add_hist : bool, default=False
    Whether to add a histogram of the `col` variable to the plot.
title : str, default=''
    Title of the plot.
val_limit : num, default=10
    Maximum number of values to test for col.
ax : Matplotlib Axis, deafault=None
    Axis to plot on.
```

```
    Returns:
    -------
    results : DataFrame
        A DataFrame with the same columns as `X`, as well as a `prob` column with
        the predicted probabilities of `clf` for each row in `X`, and a `group`
        column indicating which quantile group the row belongs to.
    """
    probs = clf.predict_proba(X)
    df = (X
          .assign(probs=probs[:,-1],
                  p_bin=lambda df_:pd.qcut(df_.probs, q=q,
                                           labels=[f'q{n}' for n in range(1,q+1)])
                  )
         )
    groups = df.groupby('p_bin')

    vals = X.loc[:,col].unique()
    if len(vals) > val_limit:
      vals = np.linspace(min(vals), max(vals), num=val_limit)
    res = []
    for name,g in groups:
      for val in vals:
        this_X = g.loc[:,X.columns].assign(**{col:val})
        q_prob = clf.predict_proba(this_X)[:,-1]
        res.append(this_X.assign(prob=q_prob, group=name))
    results = pd.concat(res, axis='index')
    if ax is None:
      fig, ax = plt.subplots(figsize=(8,4))
    if add_hist:
      back_ax = ax.twinx()
      back_ax.hist(X[col], density=True, alpha=.2)
    for name, g in results.groupby('group'):
      g.groupby(col).prob.mean().plot(ax=ax, label=name, color=color, alpha=alpha)
    if legend:
      ax.legend()
    if title:
      ax.set_title(title)
    return results
```

Let's plot the 10 quantiles for the education feature. Ideally, these lines do not cross each other. If they do cross, you might want to debug the model to make sure that it isn't overfitting on sparse data.

```
fig, ax = plt.subplots(figsize=(8,4))
quantile_ice(xgb_def, X_train, 'education', q=10, legend=False, add_hist=True, ax=ax,
             title='ICE plot for Age')
```

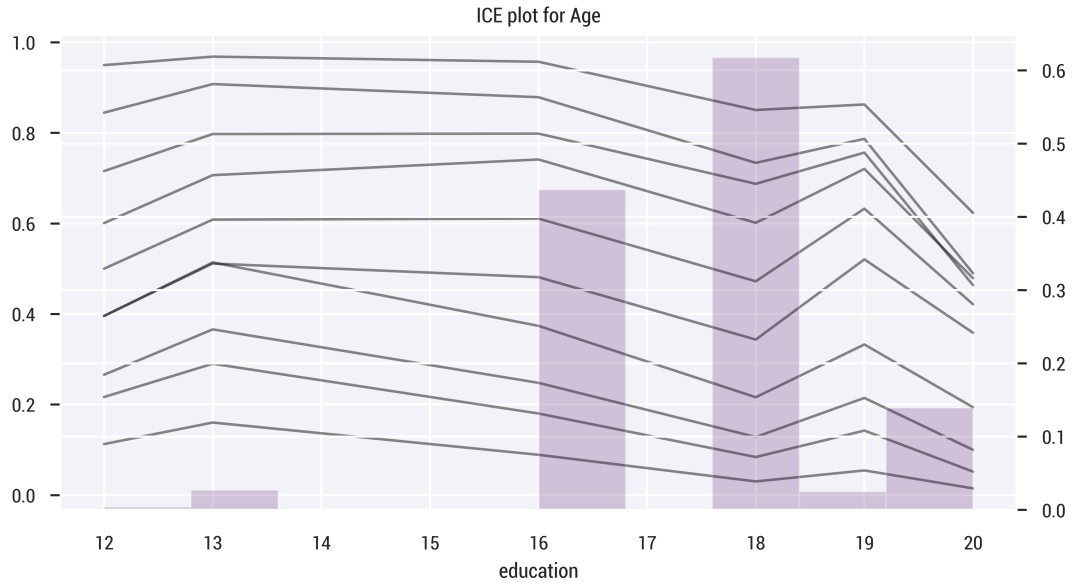


Figure 20.4: ICE plot for quantiles.

20.2 ICE Plots with SHAP

There are multiple tools to create ICE plots. If you know the magic parameters, you can get the SHAP library to create an ICE plot. It will also create the histogram.

This function is poorly documented, but I've decoded it for you. The `model` parameter needs to be a function that, given rows of data, will return probabilities. You can specify the rows to draw ice lines using the' data' parameter. You need to ensure that the `npoints` parameter is the number of unique values for a column.

```
import shap

fig, ax = plt.subplots(figsize=(8,4))

shap.plots.partial_dependence_plot(ind='education',
    model=lambda rows: xgb_def.predict_proba(rows)[:,-1],
    data=X_train.iloc[0:1000], ice=True,
    npoints=(X_train.education.nunique()),
    pd_linewidth=0, show=False, ax=ax)
ax.set_title('ICE plot (from SHAP)')
```

20.3 Partial Dependence Plots

If you set the quantile to 1 in the code above, you create a Partial Dependence Plot. It is the average of the ICE plots.

Partial Dependence Plots (PDPs) are a popular visualization technique used in machine learning to understand the relationship between input variables and the model's predicted

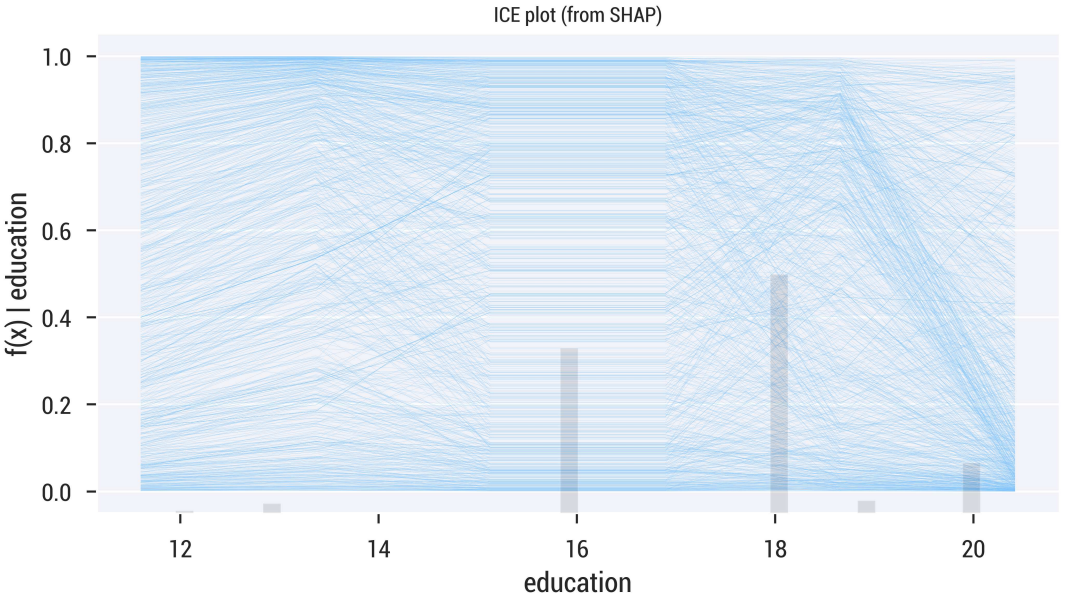


Figure 20.5: An ICE plot created whith the SHAP library.

output. PDPs illustrate the average behavior of the model for a particular input variable while holding all other variables constant. These plots allow us to identify non-linear relationships, interactions, and other important patterns in the data that are not immediately apparent from summary statistics or simple scatterplots.

```
fig, axes = plt.subplots(ncols=2, figsize=(8,4))

PartialDependenceDisplay.from_estimator(xgb_def, X_train, features=['r', 'education'],
                                        kind='average', ax=axes)
fig.tight_layout()
```

A common suggestion is to plot the PDP plot on top of a centered ICE plot. This let's us better understand the general impact of a feature.

```
fig, axes = plt.subplots(ncols=2, figsize=(8,4))

PartialDependenceDisplay.from_estimator(xgb_def, X_train, features=['r', 'education'],
                                        centered=True, kind='both',
                                        ax=axes)
fig.tight_layout()
```

Let's expore the *years_exp* and *Q1_Male* plots.

```
fig, axes = plt.subplots(ncols=2, figsize=(8,4))

PartialDependenceDisplay.from_estimator(xgb_def, X_train, features=['years_exp', 'Q1_Male'],
```

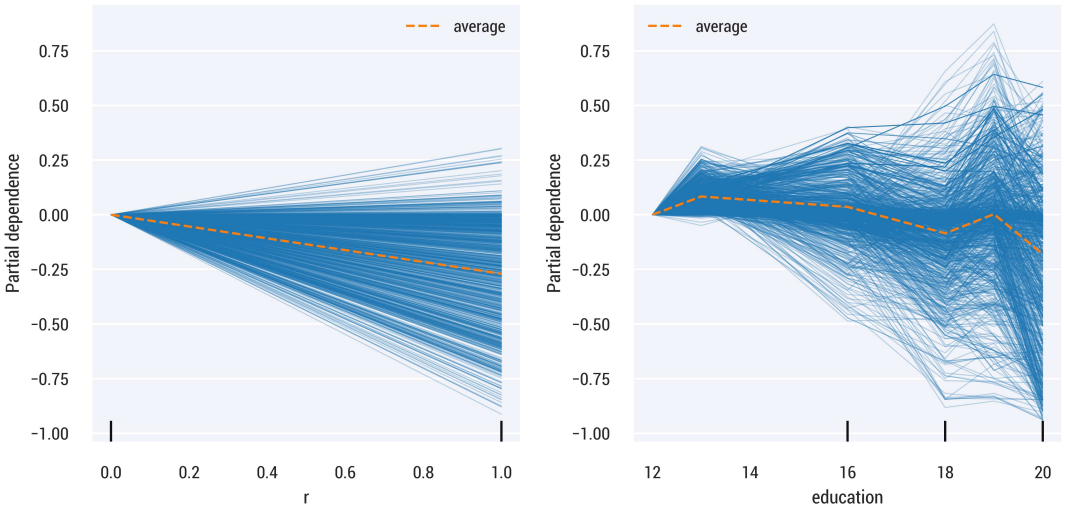


Figure 20.6: PDP and ICE plots for R and education.

```
                                    centered=True, kind='both',
                                    ax=axes)
fig.tight_layout()
```

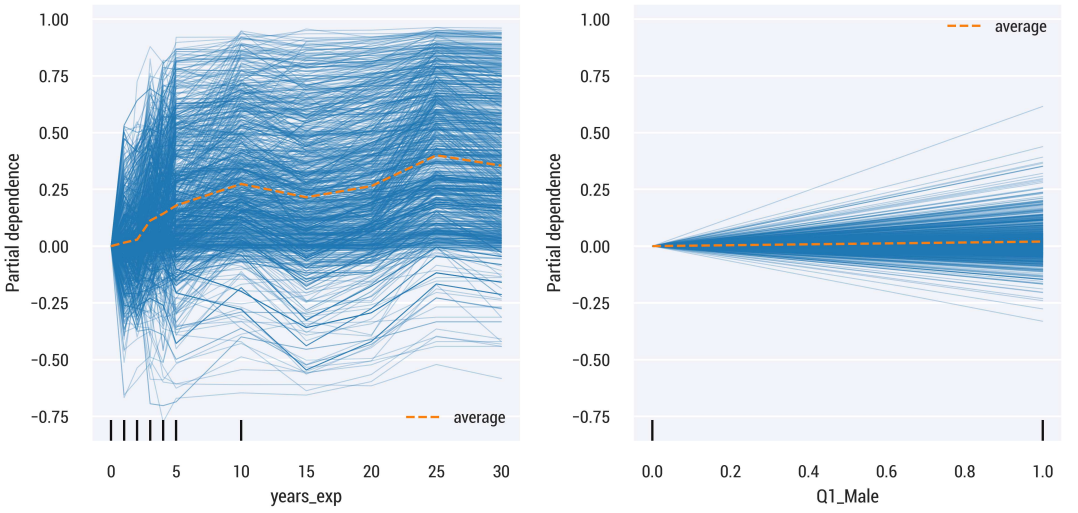


Figure 20.7: PDP and ICE plots for years_exp and Q1_Male.

It looks like *year_exp* tends to be monotonically increasing. The *Q1_Male* plot is flat for the PDP, but there is some spread for the ICE values, indicating that there is probably an interaction with other columns.

20.4 PDP with SHAP

It might not surprise you that the SHAP library can make PDP plots as well. We will create PDP plots with the aptly named `partial_dependence_plot` function. It can even add a histogram to the plot.

Let's create one for the *years_exp* feature.

```
import shap

fig, ax = plt.subplots(figsize=(8,4))

col = 'years_exp'
shap.plots.partial_dependence_plot(ind=col,
                            model=lambda rows: xgb_def.predict_proba(rows)[:,-1],
                            data=X_train.iloc[0:1000], ice=False,
                            npoints=(X_train[col].nunique()),
                            pd_linewidth=2, show=False, ax=ax)
ax.set_title('PDP plot (from SHAP)')
```

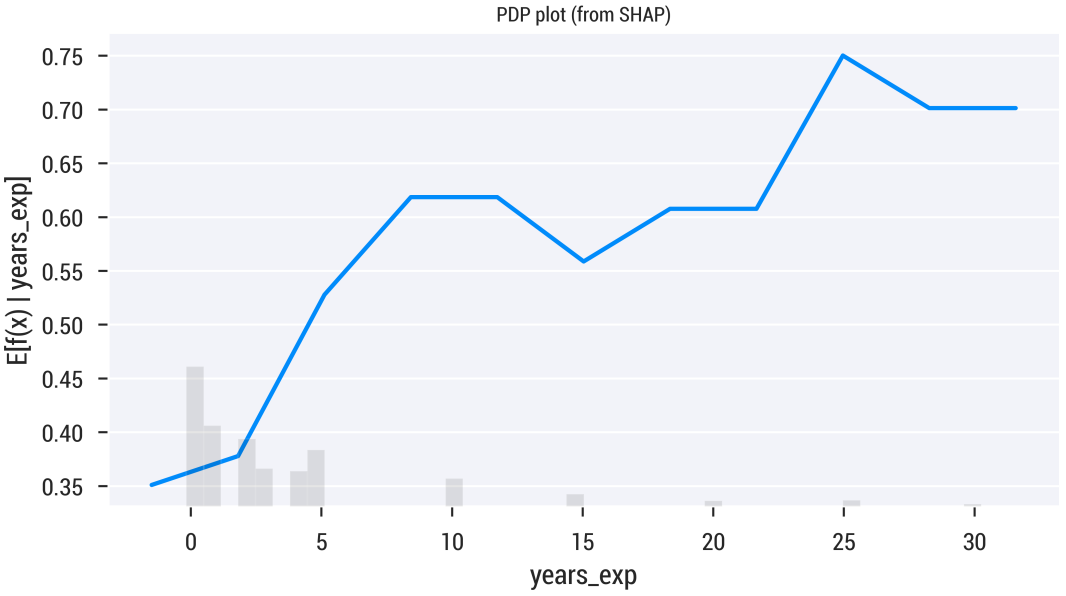


Figure 20.8: A PDP plot created with SHAP.

If we specify `ice=True`, we can plot the PDP on top of the ICE plots. In this example, we also highlight the expected value for the *years_exp* column and the expected probability for the positive label.

```
fig, ax = plt.subplots(figsize=(8,4))

col = 'years_exp'
shap.plots.partial_dependence_plot(ind=col,
```

```
                                model=lambda rows: xgb_def.predict_proba(rows)[:,-1],
                                data=X_train.iloc[0:1000], ice=True,
                                npoints=(X_train[col].nunique()),
                                model_expected_value=True,
                                feature_expected_value=True,
                                pd_linewidth=2, show=False, ax=ax)
ax.set_title('PDP plot (from SHAP) with ICE Plots')
```

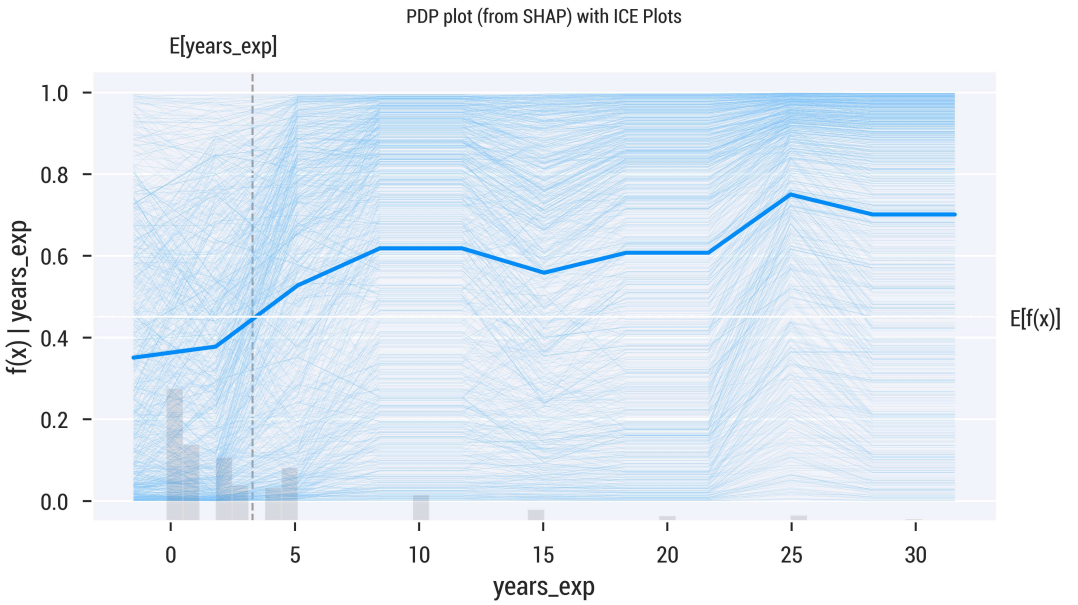


Figure 20.9: PDP and ICE plots created with SHAP.

20.5 Monotonic Constraints

PDP and ICE plots are useful visualization tools in machine learning that can help to identify *monotonic constraints* in a model.

A monotonic constraint means that as the value of a specific feature changes, the predicted outcome should either always increase or decrease. This is often the case in situations with a correlation between a feature and the outcome. The standard disclaimer that correlation is not causation applies here!

Examining the ranked correlation (Spearman) is one way to explore these relationships.

```
fig, ax = plt.subplots(figsize=(8,4))

(X_test
 .assign(target=y_test)
 .corr(method='spearman')
 .iloc[:-1]
```

```
 .loc[:,'target']
 .sort_values(key=np.abs)
 .plot.barh(title='Spearman Correlation with Target', ax=ax)
)
```

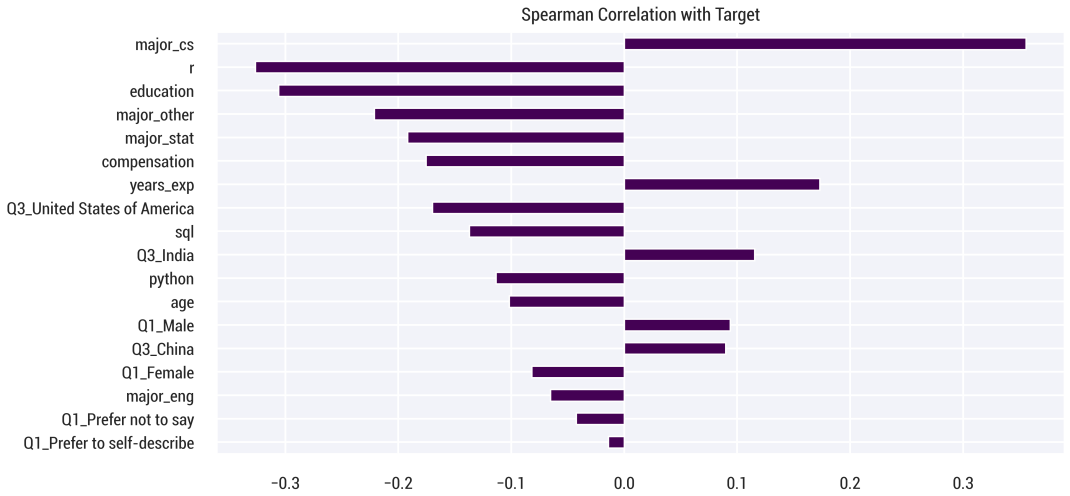


Figure 20.10: Spearman correlation with target variable. You can use this to guide you with monotonic constraints.

You can create a cutoff and explore variables above that cutoff. Assume a cutoff for an absolute value of 0.2. The *education* column is non-binary and might be a good candidate to explore.

PDP and ICE plots can also help to identify monotonic constraints by showing the relationship between a feature and the predicted outcome while holding all other features constant. If the plot shows a clear increasing or decreasing trend for a specific feature, this can indicate a monotonic constraint.

You can see the PDP for *education* in the last section.

The line appears almost monotonic if we inspect the *education* PDP plot for education. It has a small bump at 19. This plot caused me to explore the data a little more. The `tweak_kag` function converts *Professional Degree* to a value of 19 (19 years of school). Most professional degrees take 3-5 years.

Let's use some Pandas to explore what happens in education and the mean value of other columns:

```
>>> print(X_train
... .assign(target=y_train)
... .groupby('education')
... .mean()
... .loc[:, ['age', 'years_exp', 'target']]
... )

                 age  years_exp    target
education
```

```
12.0       30.428571   2.857143  0.714286
13.0       30.369565   6.760870  0.652174
16.0       25.720867   2.849593  0.605691
18.0       28.913628   3.225528  0.393474
19.0       27.642857   4.166667  0.571429
20.0       35.310638   4.834043  0.174468
```

It appears that the target value does jump at 19. Let's see how many values there are for education 19:

```
>>> X_train.education.value_counts()
18.0    1042
16.0     738
20.0     235
13.0      46
19.0      42
12.0       7
Name: education, dtype: int64
```

There aren't very many values. We can dig in a little more and inspect the raw data (remember that 19 was derived from the *Professional degree* value) to see if anything stands out:

```
>>> print(raw
... .query('Q3.isin(["United States of America", "China", "India"]) '
...        'and Q6.isin(["Data Scientist", "Software Engineer"])')
... .query('Q4 == "Professional degree"')
... .pipe(lambda df_:pd.crosstab(index=df_.Q5, columns=df_.Q6))
... )

Q6                                                Data Scientist  \
Q5
A business discipline (accounting, economics, f...              0
Computer science (software engineering, etc.)                  12
Engineering (non-computer focused)                              6
Humanities (history, literature, philosophy, etc.)              2
I never declared a major                                        0
Mathematics or statistics                                       2
Other                                                           2
Physics or astronomy                                            2

Q6                                                Software Engineer
Q5
A business discipline (accounting, economics, f...                 1
Computer science (software engineering, etc.)                     19
Engineering (non-computer focused)                                10
Humanities (history, literature, philosophy, etc.)                 0
I never declared a major                                           1
Mathematics or statistics                                          1
Other                                                              1
Physics or astronomy                                               1
```

Deciding whether to add a monotonic constraint to XGBoost depends on the specific context and goals of the model. A monotonic constraint can be added to ensure the model adheres to a particular relationship between a feature and the outcome. For example, in a scenario where we expect a clear cause-and-effect relationship between a feature and the outcome, such as in financial risk modeling, we might want to add a monotonic constraint to ensure that the model predicts an increasing or decreasing trend for that feature.

Adding a monotonic constraint can help to improve the accuracy and interpretability of the model, by providing a clear constraint on the relationship between the feature and the outcome. It can also help reduce overfitting by limiting the range of values the model can predict for a given feature.

From looking at the PDP and ICE plots it appears there is not a ton of data to not enforce a monotonic constraint for education. There is a chance that the model is overfitting the education entries with the value of 19. To simplify the model, we will add a monotonic constraint.

I will also add a constraint to the *years_exp* column. We will specify the `monotone_constraints` parameter mapping the column name to the sign of the slope of the PDP plot. Because *years_exp* increases, we will map it to 1. The *education* column is decreasing, so we map it to -1.

```
xgb_const = xgb.XGBClassifier(random_state=42,
          monotone_constraints={'years_exp':1, 'education':-1})
xgb_const.fit(X_train, y_train)
xgb_const.score(X_test, y_test)
```

It looks like the constraints improved (and simplified) our model!

I'm going to go one step further. Because the PDP line for *Q1_Male* was flat, I'm going to remove gender columns as well.

```
small_cols = ['age', 'education', 'years_exp', 'compensation', 'python', 'r', 'sql',
              #'Q1_Male', 'Q1_Female', 'Q1_Prefer not to say',
              #'Q1_Prefer to self-describe',
              'Q3_United States of America', 'Q3_India',
              'Q3_China', 'major_cs', 'major_other', 'major_eng', 'major_stat']
xgb_const2 = xgb.XGBClassifier(random_state=42,
          monotone_constraints={'years_exp':1, 'education':-1})
xgb_const2.fit(X_train[small_cols], y_train)
```

Let's look at the score:

```
>>> xgb_const2.score(X_test[small_cols], y_test)
0.7569060773480663
```

Slightly better! And simpler!

It looks like these constraints improved our model (and also simplified it). Another way to evaluate this is to look at the feature importance values. Our default model focuses on the *R* column now. The constrained model will hopefully spread out the feature importance more.

```
fig, ax = plt.subplots(figsize=(8,4))
(pd.Series(xgb_def.feature_importances_, index=X_train.columns)
 .sort_values()
 .plot.barh(ax=ax)
)
```

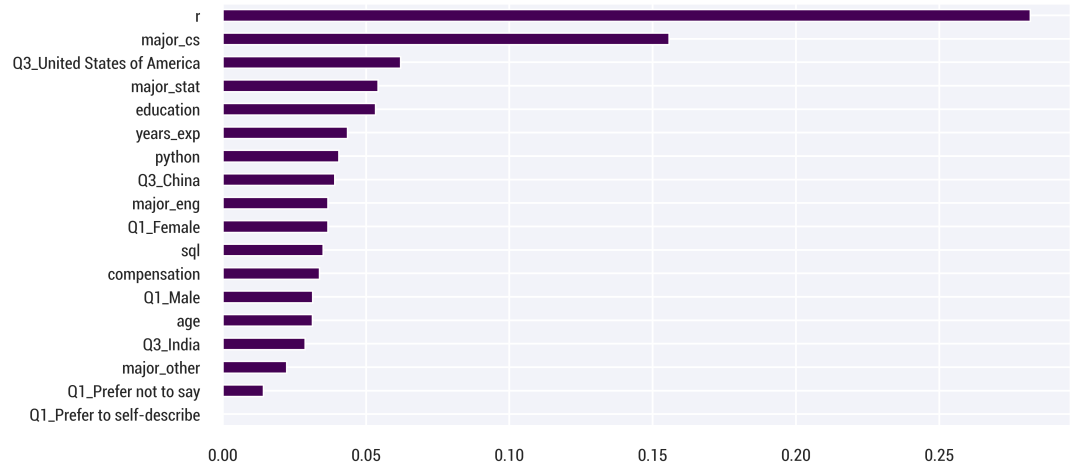


Figure 20.11: Unconstrained feature importance. Notice that the R column is driving our model right now. We would like to distribute the importance if possible.

It appears that the feature importance values from the constrained model are slightly more evenly disbursed indicating that that model will pay less attention to the *R* column.

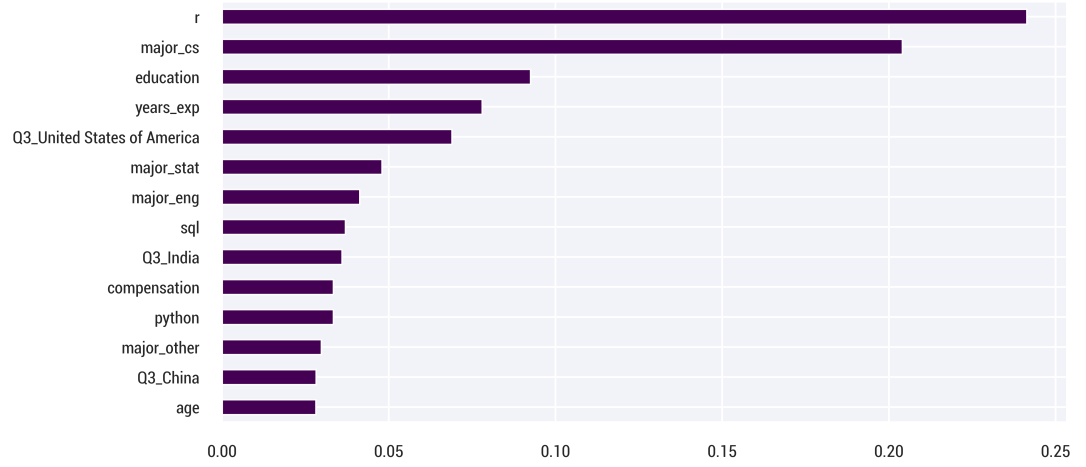


Figure 20.12: Constrained feature importance. Notice that the R column has less impact and the other columns have more.

```
fig, ax = plt.subplots(figsize=(8,4))
(pd.Series(xgb_const2.feature_importances_, index=small_cols)
 .sort_values()
 .plot.barh(ax=ax)
)
```

20.6 Calibrating a Model

This section will look at fine-tuning our model with *calibration*. Calibration refers to adjusting a model's output to better align with the actual probabilities of the target variable. If we want to use the probabilities (from `.predict_proba`) and not just the target, we will want to calibrate our model.

With XGBoost, the probability output often does not correspond to the actual probabilities of the target variable. XGBoost models tend to produce predicted probabilities biased towards the ends of the probability range, meaning they often overestimate or underestimate the actual probabilities. This will lead to poor performance on tasks that require accurate probability estimates, such as ranking, threshold selection, and decision-making.

Calibrating an XGBoost model involves post-processing the predicted probabilities of the model using a calibration method, such as Platt scaling or isotonic regression. These methods map the model's predicted probabilities to calibrated probabilities that better align with the actual probabilities of the target variable.

Let's calibrate the model using scikit-learn.

Two calibration types are available, *sigmoid* and *isotonic*. Sigmoid calibration involves fitting a logistic regression model to the predicted probabilities of a binary classifier and transforming the probabilities using the logistic function. Isotonic calibration consists in fitting a non-parametric monotonic function to the predicted probabilities, ensuring that the function increases with increasing probabilities.

Both techniques can improve the reliability of the probabilistic predictions of a model, but they differ in their flexibility and interpretability. Sigmoid calibration is a simple and computationally efficient method that can be easily implemented, but it assumes a parametric form for the calibration function and may not capture more complex calibration patterns. Isotonic calibration, on the other hand, is a more flexible and data-driven method that can capture more complex calibration patterns. Still, it may require more data and computational resources to implement.

We will try both and compare the results. We need to provide the `CalibratedClassifierCV` with our existing model and the `method` parameter. I'm using `cv=prefit` because I already fit the model. Then we call the `.fit` method.

```
from sklearn.calibration import CalibratedClassifierCV

xgb_cal = CalibratedClassifierCV(xgb_def, method='sigmoid', cv='prefit')
xgb_cal.fit(X_test, y_test)

xgb_cal_iso = CalibratedClassifierCV(xgb_def, method='isotonic', cv='prefit')
xgb_cal_iso.fit(X_test, y_test)
```

20.7 Calibration Curves

We will visualize the results with a *calibration curve*.

A calibration curve is a graphical representation of the relationship between predicted probabilities and the actual frequencies of events in a binary classification problem. The curve plots the predicted probabilities (x-axis) against the observed frequencies of the positive class (y-axis) for different probability thresholds.

The calibration curve visually assesses how well a classification model's predicted probabilities match the true probabilities of the events of interest. Ideally, a well-calibrated model should have predicted probabilities that match the actual probabilities of the events, meaning that the calibration curve should be close to the diagonal line.

If the calibration curve deviates from the diagonal line, it suggests that the model's predicted probabilities are either overconfident or underconfident. An overconfident model predicts high probabilities for events that are unlikely to occur, while an underconfident model predicts low probabilities for events that are likely to occur.

```
from sklearn.calibration import CalibrationDisplay
from matplotlib.gridspec import GridSpec
fig = plt.figure(figsize=(8,6))
gs = GridSpec(4, 3)
axes = fig.add_subplot(gs[:2, :3])
display = CalibrationDisplay.from_estimator(xgb_def, X_test, y_test,
                                            n_bins=10, ax=axes)
disp_cal = CalibrationDisplay.from_estimator(xgb_cal, X_test, y_test,
                                     n_bins=10,ax=axes, name='sigmoid')
disp_cal_iso = CalibrationDisplay.from_estimator(xgb_cal_iso, X_test, y_test,
                                     n_bins=10, ax=axes, name='isotonic')
row = 2
col = 0
ax = fig.add_subplot(gs[row, col])
ax.hist(display.y_prob, range=(0,1), bins=20)
ax.set(title='Default', xlabel='Predicted Prob')
ax2 = fig.add_subplot(gs[row, 1])
ax2.hist(disp_cal.y_prob, range=(0,1), bins=20)
ax2.set(title='Sigmoid', xlabel='Predicted Prob')
ax3 = fig.add_subplot(gs[row, 2])
ax3.hist(disp_cal_iso.y_prob, range=(0,1), bins=20)
ax3.set(title='Isotonic', xlabel='Predicted Prob')
fig.tight_layout()
```

The calibration curve suggests that our default model does a respectable job. It is tracking the diagonal pretty well. But our calibrated models look like they track the diagonal better.

The histograms show the distribution of the default model and the calibrated models.

Let's look at the score of our calibrated models. It looks like they perform slightly better (at least for accuracy).

```
>>> xgb_cal.score(X_test, y_test)
0.7480662983425415

>>> xgb_cal_iso.score(X_test, y_test)
0.7491712707182321

>>> xgb_def.score(X_test, y_test)
0.7458563535911602
```

20.8 Summary

This chapter explored ICE and PDP plots. Then we looked at monotonic constraints and model calibration. ICE and PDP plots are two visualization techniques used to interpret the output of XGBoost models. ICE plots show the predicted outcome of an individual instance as a function of a single feature. In contrast, PDP plots show the average predicted outcome across all instances as a function of a single feature. Monotonic constraints can be added

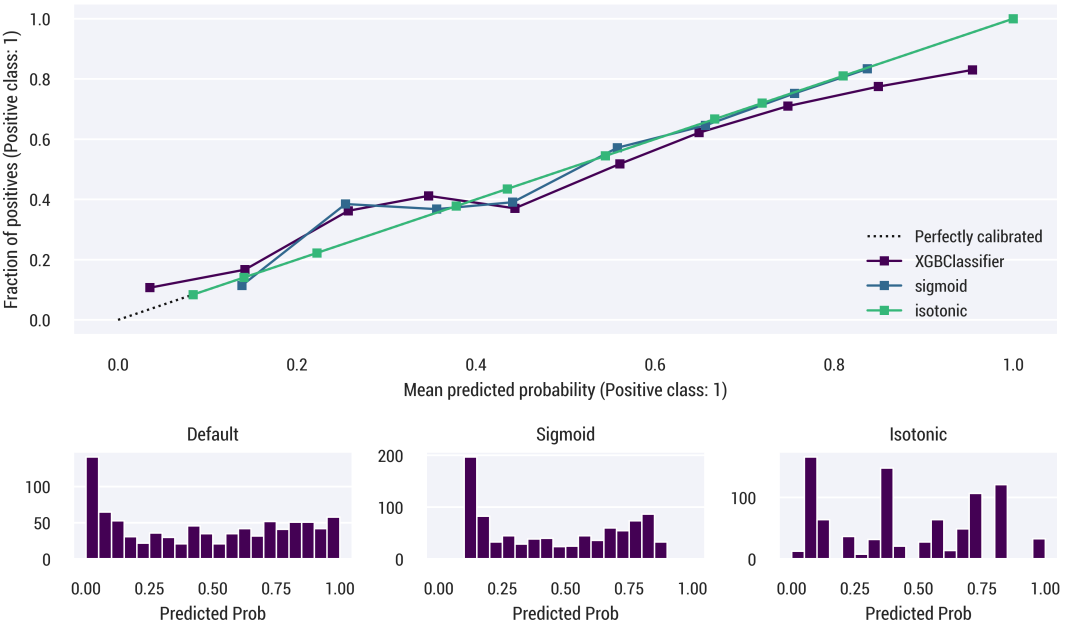


Figure 20.13: Calibration curves for the default and calibrated models. Models that track the diagonal tend to return probabilities that reflect the data.

to XGBoost models to ensure that the model's predictions increase or decrease monotonically with increasing feature values. This can simplify the model to prevent overfitting. Calibration techniques such as sigmoid or isotonic calibration can be used to improve the reliability of the model's probabilistic predictions.

20.9 Exercises

1. How can ICE and PDP plots identify non-linear relationships between features and the predicted outcome in an XGBoost model?
2. How can ICE and PDP plots be used to compare the effects of different features on the predicted outcome in an XGBoost model?
3. Can ICE and PDP plots visualize interactions between features in an XGBoost model? If so, how?
4. What are monotonic constraints in XGBoost, and how do they impact the model's predictions?
5. How can monotonic constraints be specified for individual features in an XGBoost model?
6. What are some potential benefits of adding monotonic constraints to an XGBoost model, and in what situations are they particularly useful?
7. How can the calibration curve be used to assess the calibration of an XGBoost model's predicted probabilities?
8. How can calibration techniques help improve the reliability of an XGBoost model's probabilistic predictions in practical applications?

Chapter 21

Serving Models with MLFlow

In this chapter, we will explore the MLFlow library.

MLflow is an open-source library for managing the end-to-end machine learning lifecycle. It provides tools for tracking experiments, packaging machine learning models, and deploying models to production. MLflow is particularly useful for XGBoost because it allows users to track and compare the performance of different XGBoost models and quickly deploy the best-performing model to production.

One of the key features of MLflow is its experiment-tracking capability. This allows users to log various metrics, such as model accuracy or training time, and compare the results of different experiments. This can be useful for identifying the most effective hyperparameters for an XGBoost model and choosing the best-performing model.

MLflow also includes a model packaging feature, which makes it easy to package and deploy XGBoost models. This makes it possible to deploy XGBoost models to production environments and integrate them with other machine learning systems.

21.1 Installation and Setup

MLFlow is a third-party library. Make sure you install it.

Let's show how to use MLFlow with the model we have been developing. We will then serve the model using MLFlow.

Here are the imports that we will need:

```
%matplotlib inline

from feature_engine import encoding, imputation
from hyperopt import fmin, tpe, hp, STATUS_OK, Trials
import matplotlib.pyplot as plt
import mlflow
import numpy as np
import pandas as pd
from sklearn import base, metrics, model_selection, \
   pipeline, preprocessing
from sklearn.metrics import accuracy_score, roc_auc_score
import xgboost as xgb


import urllib
import zipfile
```

Let's load the raw data.
This code will create a pipeline and prepare the training and testing data.

```
import pandas as pd
from sklearn import model_selection, preprocessing
import xg_helpers as xhelp


url = 'https://github.com/mattharrison/datasets/raw/master/data/'\
    'kaggle-survey-2018.zip'
fname = 'kaggle-survey-2018.zip'
member_name = 'multipleChoiceResponses.csv'

raw = xhelp.extract_zip(url, fname, member_name)
## Create raw X and raw y
kag_X, kag_y = xhelp.get_rawX_y(raw, 'Q6')

## Split data
kag_X_train, kag_X_test, kag_y_train, kag_y_test = \
    model_selection.train_test_split(
        kag_X, kag_y, test_size=.3, random_state=42, stratify=kag_y)

## Transform X with pipeline
X_train = xhelp.kag_pl.fit_transform(kag_X_train)
X_test = xhelp.kag_pl.transform(kag_X_test)

## Transform y with label encoder
label_encoder = preprocessing.LabelEncoder()
label_encoder.fit(kag_y_train)
y_train = label_encoder.transform(kag_y_train)
y_test = label_encoder.transform(kag_y_test)

# Combined Data for cross validation/etc
X = pd.concat([X_train, X_test], axis='index')
y = pd.Series([*y_train, *y_test], index=X.index)
```

This code uses hypopt to train the model. It has been extended to log various metrics in MLFlow while training.

```
from hyperopt import fmin, tpe, hp, STATUS_OK, Trials
import mlflow
from sklearn import metrics
import xgboost as xgb

ex_id = mlflow.create_experiment(name='ex3', artifact_location='ex2path')
mlflow.set_experiment(experiment_name='ex3')
with mlflow.start_run():
    params = {'random_state': 42}
    rounds = [{'max_depth': hp.quniform('max_depth', 1, 12, 1),  # tree
               'min_child_weight': hp.loguniform('min_child_weight', -2, 3)},
              {'subsample': hp.uniform('subsample', 0.5, 1),   # stochastic
               'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 1)},
```

```
            {'gamma': hp.loguniform('gamma', -10, 10)}, # regularization
            {'learning_rate': hp.loguniform('learning_rate', -7, 0)} # boosting
    ]

    for round in rounds:
        params = {**params, **round}
        trials = Trials()
        best = fmin(fn=lambda space: xhelp.hyperparameter_tuning(
                space, X_train, y_train, X_test, y_test),
            space=params,
            algo=tpe.suggest,
            max_evals=10,
            trials=trials,
            timeout=60*5 # 5 minutes
        )
        params = {**params, **best}
        for param, val in params.items():
            mlflow.log_param(param, val)
        params['max_depth'] = int(params['max_depth'])
        xg = xgb.XGBClassifier(eval_metric='logloss', early_stopping_rounds=50, **params)
        xg.fit(X_train, y_train,
               eval_set=[(X_train, y_train),
                         (X_test, y_test)
                        ]
              )
        for metric in [metrics.accuracy_score, metrics.precision_score, metrics.recall_score,
                       metrics.f1_score]:
            mlflow.log_metric(metric.__name__, metric(y_test, xg.predict(X_test)))

    model_info = mlflow.xgboost.log_model(xg, artifact_path='model')
```

At the end of this code, you'll notice that we store the results of calling `log_model` in `model_info`. The `log_model` function stores the xgboost model as an artifact. It returns an object that has metadata about the model that we created.

If you inspect the `ex_id` variable and the `.run_id` attribute, they will point to a directory that stores information about the model.

```
>>> ex_id
'172212630951564101'

>>> model_info.run_id
'263b3e793f584251a4e4cd1a2d494110'
```

The directory structure looks like this:

```
mlruns/172212630951564101/263b3e793f584251a4e4cd1a2d494110
├── artifacts
├── meta.yaml
├── metrics
│   ├── accuracy_score

│   ├── f1_score
```

```
│   ├── precision_score
│   └── recall_score
├── params
│   ├── colsample_bytree
│   ├── gamma
│   ├── learning_rate
│   ├── max_depth
│   ├── min_child_weight
│   ├── random_state
│   └── subsample
└── tags
    ├── mlflow.log-model.history
    ├── mlflow.runName
    ├── mlflow.source.name
    ├── mlflow.source.type
    └── mlflow.user
```

We can launch a server to run the model if we know the artifact.

21.2 Inspecting Model Artifacts

You can tell MLFlow to launch a service to inspect the artifacts of training the model.
From the command line, use the following command:

```
mlflow ui
```

Then go to the URL localhost:5000
You should see a site that looks like this.

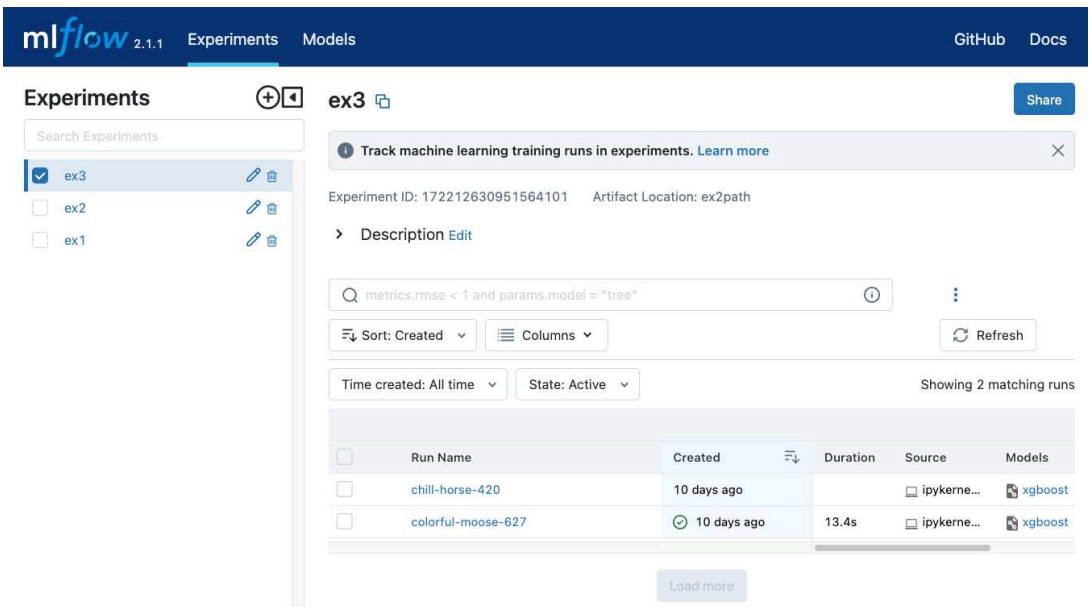


Figure 21.1: MLFlow UI Homepage

It will create unique identifiers of the runs like *colorful-moose-627*. You can inspect the value of `tags/mlflow.runName` file to find the run name. MLFlow automatically names your models with unique ids.

21.3 Running A Model From Code

When you click on a finished model, MLFlow will give you the code to make predictions. You load the model using the `load_model` function and pass in the *run id*.

```
import mlflow
logged_model = 'runs:/ecc05fedb5c942598741816a1c6d76e2/model'

# Load model as a PyFuncModel.
loaded_model = mlflow.pyfunc.load_model(logged_model)
```

This code above is available from the website found from launching the command line service. If you click on the model folder icon, it will show how to make a prediction with Pandas (like above) or with Spark.

Once you have loaded the model, you can make predictions. Let's predict the first row of the test data:

```
>>> loaded_model.predict(X_test.iloc[[0]])
array([1])
```

21.4 Serving Predictions

MLFlow also creates an endpoint that we can query for predictions.

If you have the uuid, you can launch a service for it. By default, it will use `pyenv` to create a fresh environment to run the service. I'm passing `--env-manager local` to bypass that and use my local environment:

```
mlflow models serve -m mlruns/172212630951564101/ \
  263b3e793f584251a4e4cd1a2d494110/artifacts/model \
  -p 1234 --env-manager local
```

The above command starts a service on port 1234. If you hit the URL and port in a web browser, it will throw an error because it is expecting an HTTP POST command. The next section will show you how to query the service.

MLFlow also enables you to serve the same model from Azure, AWS Sagemaker, or Spark. Explore the documentation for MLFlow about the built-in deployment tools for more details.

21.5 Querying from the Command Line

You can also query models from the command line with `curl`.

On UNIX systems, you can use the `curl` command. It look like this (make sure to replace `$URL` and `$JSON_DATA` with the appropriate values):

```
curl $URL -X POST -H "Content-Type:application/json" --data $JSON_DATA
```

You will need to create JSON_DATA data with this format. I'm showing it on multiple lines, but you will put it on a single line.:

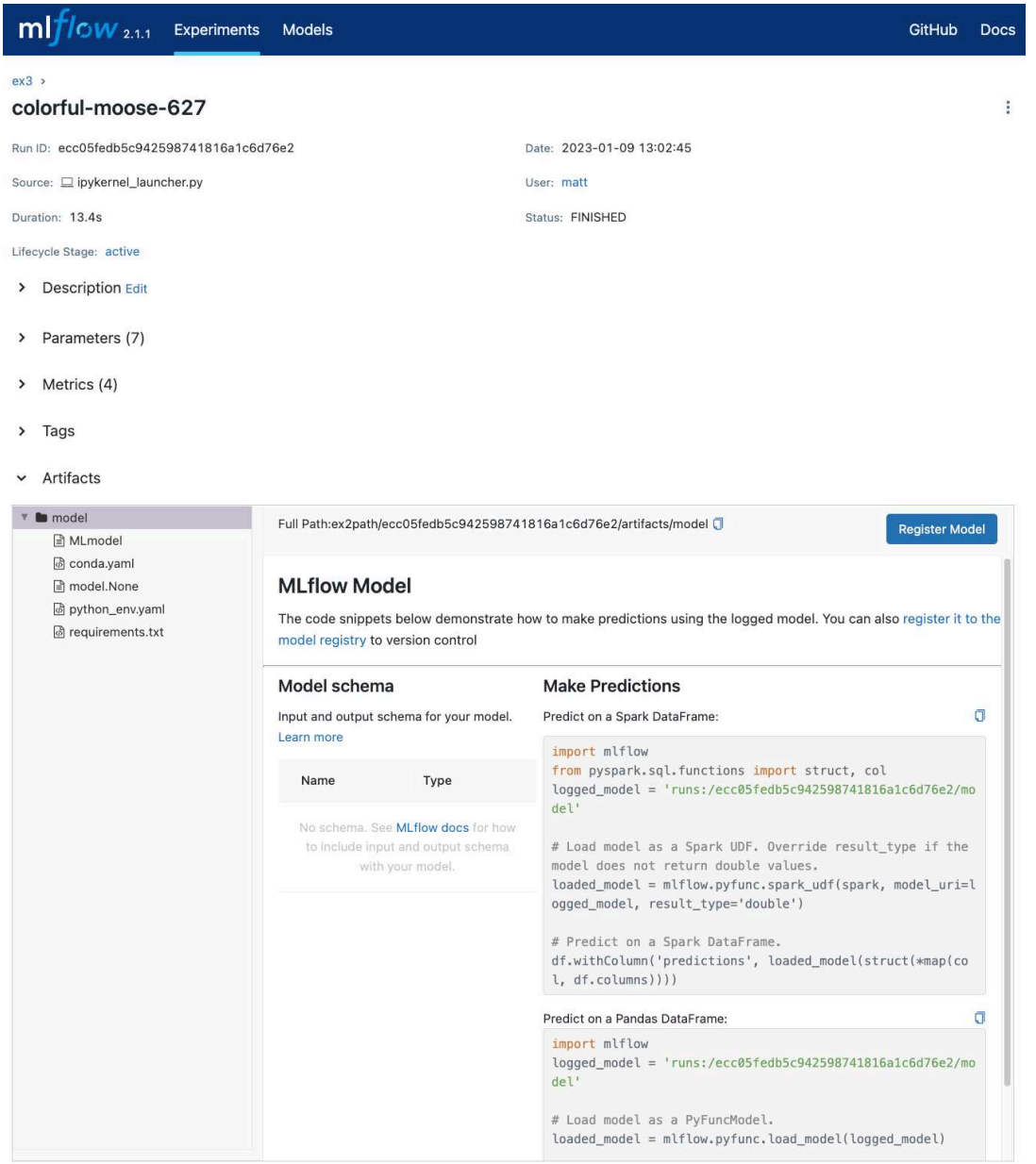


Figure 21.2: Inspecting a run of an experiment. You can expand the *Parameters* and the *Metrics* on the left-hand side. In the lower right is the code to run a model.

```
{'dataframe_split':
  {'columns': ['col1', 'col2'],
   'data': [[22, 16.0],
            [25, 18.0]]}
}
```

If you don't want to create the JSON manually, you can use Pandas to make it. If you have a Dataframe with the data, use the Pandas `.to_json` method and set `orient='split'`.

```
>>> X_test.head(2).to_json(orient='split', index=False)
'{"columns":["age","education","years_exp","compensation",
"python","r","sql","Q1_Male","Q1_Female","Q1_Prefer not to say",
"Q1_Prefer to self-describe","Q3_United States of America",
"Q3_India","Q3_China","major_cs","major_other","major_eng",
"major_stat"],"data":[[22,16.0,1.0,0,1,0,0,1,0,0,0,0,1,0,1,0,
0,0],[25,18.0,1.0,70000,1,1,0,1,0,0,0,1,0,0,0,1,0,0]]}'
```

This is a JSON string, and we need a Python dictionary so that we can embed this in another dictionary. Consider this value to be `DICT`. We must place it in another dictionary with the key *dataframe_split* : `{'dataframe_split: DICT}`. We will use the `json.loads` function to create a dictionary from the string. (We can't use the Python string because the quotes are incorrect for JSON.)

Here is the JSON data we need to insert into the dictionary:

```
>>> import json
>>> json.loads(X_test.head(2).to_json(orient='split', index=False))
{'columns': ['age',
  'education',
  'years_exp',
  'compensation',
  'python',
  'r',
  'sql',
  'Q1_Male',
  'Q1_Female',
  'Q1_Prefer not to say',
  'Q1_Prefer to self-describe',
  'Q3_United States of America',
  'Q3_India',
  'Q3_China',
  'major_cs',
  'major_other',
  'major_eng',
  'major_stat'],
 'data': [[22, 16.0, 1.0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0],
  [25, 18.0, 1.0, 70000, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0]]}
```

Then you need to nest this in another dictionary under the key `'dataframe_split'`.

```
>>> {'dataframe_split': json.loads(X_test.head(2).to_json(orient='split',
...                                                       index=False))}
{'dataframe_split': {'columns': ['age',
   'education',
```

```
   'years_exp',
   'compensation',
   'python',
   'r',
   'sql',
   'Q1_Male',
   'Q1_Female',
   'Q1_Prefer not to say',
   'Q1_Prefer to self-describe',
   'Q3_United States of America',
   'Q3_India',
   'Q3_China',
   'major_cs',
   'major_other',
   'major_eng',
   'major_stat'],
  'data': [[22, 16.0, 1.0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0],
   [25, 18.0, 1.0, 70000, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0]]}}
```

Finally, we need to convert the Python dictionary back to a valid JSON string. We will use the `json.dumps` function to do that:

```
>>> import json
>>> post_data = json.dumps({'dataframe_split': json.loads(
...     X_test.head(2).to_json(orient='split', index=False))})
>>> post_data
'{"dataframe_split": {"columns": ["age", "education",
"years_exp", "compensation", "python", "r", "sql", "Q1_Male",
"Q1_Female", "Q1_Prefer not to say", "Q1_Prefer to self-describe",
"Q3_United States of America", "Q3_India", "Q3_China", "major_cs",
"major_other", "major_eng", "major_stat"], "data": [[22, 16.0,
1.0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0], [25, 18.0, 1.0,
70000, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0]]}}'
```

I'll make a function that transforms a dataframe into this format to avoid repetition.

```
def create_post_data(df):
    dictionary = json.loads(df
       .to_json(orient='split', index=False))
    return json.dumps({'dataframe_split': dictionary})
```

Let's try out the function:

```
>>> post_data = create_post_data(X_test.head(2))
>>> print(post_data)
{"dataframe_split": {"columns": ["age", "education", "years_exp", "compensation",
   "python", "r", "sql", "Q1_Male", "Q1_Female", "Q1_Prefer not to say",
   "Q1_Prefer to self-describe", "Q3_United States of America", "Q3_India", "Q3_China",
   "major_cs", "major_other", "major_eng", "major_stat"],
 "data": [[22, 16.0, 1.0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0],
          [25, 18.0, 1.0, 70000, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0]]}}
```

In Jupyter, we can use Python variables in shell commands. To tell Jupyter to use the value of the variable, we need to stick a dollar sign ($) in front of the variable. However, when we throw the post_data variable into the curl command in Jupyter, it fails.

```
!curl http://127.0.0.1:1234/invocations -X POST -H \
    "Content-Type:application/json" --data $post_data
```

It fails with this error:

```
curl: (3) unmatched brace in URL position 1:
{columns:
 ^
```

This is because we need to wrap the contents of the post_data variable with single quotes. I'm going to stick the quotes directly into the Python string.

```
>>> quoted = f"'{post_data}'"
>>> quoted
'\'{"dataframe_split": {"columns": ["age", "education",
"years_exp", "compensation", "python", "r", "sql", "Q1_Male",
"Q1_Female", "Q1_Prefer not to say", "Q1_Prefer to self-describe",
"Q3_United States of America", "Q3_India", "Q3_China", "major_cs",
"major_other", "major_eng", "major_stat"], "data": [[22, 16.0, 1.0,
0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0], [25, 18.0, 1.0,
70000, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0]]}}\''
```

Let's stick the capability to "quote" the data into the create_post_data function.

```
def create_post_data(df, quote=True):
    dictionary = {'dataframe_split': json.loads(df
       .to_json(orient='split', index=False))}
    if quote:
        return f"'{dictionary}'"
    else:
        return dictionary

quoted = create_post_data(X_test.head(2))
```

Let's try it now:

```
!curl http://127.0.0.1:1234/invocations -x post -h \
        "content-type:application/json" --data $quoted
```

This returns the JSON:

```
{"predictions": [1, 0]}
```

Indicating that the first row is a software engineer and the second is a data scientist.

21.6 Querying with the Requests Library

I'll show you how to query the service from python using the `requests` library.

This code uses the `requests` library to make a post request. It will predict the first two test rows. it uses the pandas `.to_json` method and requires setting `orient='split'`. it returns 1 (software engineer) for the first row and 0 (data scientist) for the second.

In this case, because we are sending the json data as a dictionary and not a quoted string, we will pass in `quote=false`.

```
>>> import requests as req
>>> import json

>>> r = req.post('http://127.0.0.1:1234/invocations',
...     json=create_post_data(x_test.head(2), quote=False))
>>> print(r.text)
{"predictions": [1, 0]}
```

Again, the `.text` result indicates that the web service predicted software engineer for the first row and data scientist for the second.

21.7 Building with Docker

Docker is a platform that enables developers to package, deploy, and run applications in *containers*. A container is a lightweight, stand-alone executable package that includes everything needed to run a piece of software, including the code, runtime, system tools, libraries, and settings.

Docker provides consistent and reproducible development and deployment environments across different platforms, making it an efficient and reliable tool for deploying applications, including machine learning models. It also makes it simple to scale the deployment by running multiple container instances.

If you want to build a Docker image of the model and the service, you can use this command. (You will need to point to the correct model directory and replace `MODELNAME` with the name you want to use for the Docker image.)

```
mlflow models build-docker -m mlruns/172212630951564101/ \
  263b3e793f584251a4e4cd1a2d494110/artifacts/model \
  -n MODELNAME
```

Note that you make sure you install the extra packages when installing MLFlow to get Docker capabilities:

```
pip install mlflow[extras]
```

After you have built the Docker image, you can run it with:

```
docker run --rm -p 5001:8080 MODELNAME
```

This will run the application locally on port 5001.

You can query it from the command line or from code, as we illustrated previously:

```
curl http://127.0.0.1:5001/invocations -X POST -H \
"Content-Type:application/json" --data '{"dataframe_split": \
{"columns": ["age", "education", "years_exp", "compensation", \
"python", "r", "sql", "Q1_Male", "Q1_Female", \
"Q1_Prefer not to say", "Q1_Prefer to self-describe", \
"Q3_United States of America", "Q3_India", "Q3_China", \
"major_cs", "major_other", "major_eng", "major_stat"], \
"data": [[22, 16.0, 1.0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, \
1, 0, 0, 0], [25, 18.0, 1.0, 70000, 1, 1, 0, 1, 0, 0, 0, 1,\
0, 0, 0, 1, 0, 0]]}}'
```

21.8 Conclusion

This chapter introduced the MLFlow library, an open-source library for managing the end-to-end machine learning lifecycle. It provides tools for tracking experiments, packaging machine learning models, and deploying models to production. We showed how to use the MLFlow library with the XGBoost library, how to inspect the artifacts of training the model, how to run a model from code, how to make predictions with the loaded model and launch the service, and how to use Docker with MLFlow.

21.9 Exercises

1. What is the MLFlow library, and what are its main features?
2. How does MLFlow help track and compare the performance of different XGBoost models?
3. How do you use MLFlow to package and deploy XGBoost models?
4. How do you create a Docker image from MLFlow?

Chapter 22

Conclusion

Well, folks. We are at the end. I hope you enjoyed this journey. Throughout this journey, we have explored the depths of this powerful algorithm and learned how to build highly accurate classification models for real-world applications. We have covered everything from data preparation and feature engineering to hyperparameter tuning and model evaluation. To use a cliche…

But the journey doesn't end here. It's just the beginning. The knowledge you have gained from this book can be applied to a wide range of classification problems, from predicting customer churn in business to diagnosing medical conditions in healthcare.

So, what are you waiting for? Please take what you have learned and put it into practice. Practice is the best way to make sure you are absorbing this content. And please let me know. I'd love to hear what you used this for.

Please reach out if you need help with XGBoost and want consulting or training for your team. You can find my offerings at www.metasnake.com.

If you are a teacher using this book as a textbook, don't hesitate to contact me, I would love to present on XGBoost to your class.

22.1 One more thing

As an independent self-published author, I know firsthand how challenging it can be to share this content. Without the backing of a major publishing house or marketing team, I rely on the support of my readers to help spread the word about my work. That's why I'm urging you to take a few minutes out of your day to post a review and share my book with your friends and family.

If you enjoyed this book, posting an honest review is the best way you can say thanks for the countless hours I've poured into my work. If you do post a review, please let me know. I'd love to link to it on the homepage of the book. Your support can help me reach a wider audience. So please, if you've enjoyed my book, consider leaving a review and sharing it with others. Your support means the world to me and will impact my future endeavors.

About the Author

Matt Harrison is a renowned corporate trainer specializing in Python and Pandas. With over a decade of experience in the field, he has trained thousands of professionals and helped them develop their skills in data analysis, machine learning, and automation using Python.

Matt authored several best-selling Python and data analysis books, including *Illustrated Guide to Python 3*, *Effective Pandas*, *Machine Learning Pocket Reference*, and more. He is a frequent speaker at industry conferences and events, where he shares his expertise and insights on the latest trends and techniques in data analysis and Python programming.

Technical Editors

Dr. Ronald Legere earned his Ph.D. in Physics from Yale University in 1998, with a focus on laser cooling and cold atom collisions. He then served as a Post Doc in Quantum Information Theory at Caltech and later joined MIT Lincoln Laboratory in 2000 as technical staff. There, he managed a team of analysts and support staff providing systems analysis and cost-benefit analysis to various Federal programs. In 2013, Dr. Legere became CEO of Legere Pharmaceuticals, leveraging his technical and leadership experience to improve business systems and regulatory compliance. In 2021, after selling Legere, he founded Rational Pursuit LLC to provide scientific, data-driven insights to businesses and organizations.

Edward Krueger is an expert in data science and scientific applications. As the proprietor of Peak Values Consulting, he specializes in developing and deploying simulation, optimization, and machine learning models for clients across various industries, from programmatic advertising to heavy industry.

In addition to his consulting work, Krueger is also an Adjunct Assistant Professor in the Economics Department at The University of Texas at Austin. He teaches programming, data science, and machine learning to master's students.

Alex Rook is a data scientist and Artificial Engineer based in North Carolina. He has degrees in Cognitive Science and Philosophy from the University of Virginia. He likes designing robust systems that look and feel good, function perfectly, and answer questions of high importance to multiple stakeholders who often have competing - yet valid - interests in their organization. He has extensive experience in non-financial risk management and mitigation.

Index

., 172
$, 197
'error', 71
**, 47, 106
.add, 19, 144
.assign, 154
.background_gradient, 15
.barh, 137
.bar, 17
.base_values, 154
.best_params_, 47, 81
.classes_, 34
.coef_, 134
.corr, 15, 162
.count, 146
.cv_results_, 48
.estimators_, 52
.evals_result, 68
.feature_importances_, 135, 137
.fit_transform, 11, 31
.fit, 27, 31, 61
.from_estimator, 120, 176, 185
.get_params, 43, 52, 53
.groupby, 181
.inverse_transform, 34
.loc, 26
.mean, 48
.on, 144
.pipe, 146
.plot.bar, 17
.plot.scatter, 94
.plot_tree, 28, 40, 52, 53, 62
.plot, 144
.predict_proba, 63, 119, 124, 185
.predict, 64
.query, 10, 27, 182
.rename, 154
.run_id, 191
.scale, 21, 144
.scatter, 19, 94
.score, 32, 45, 61, 115
.set_sticky, 15
.set_visible, 146
.set_xlabel, 146
.set_xticks, 146
.set_yticks, 146
.spines, 146
.text, 146
.tight_layout, 176
.to_json, 198
.transform, 11, 34
.unstack, 145, 146
.value_counts, 182
.values, 152
.view, 135
.view (dtreeviz), 54
.violinplot, 96
BaseEstimator, 10
CalibratedClassifierCV, 185
CalibrationDisplay, 185
ConfusionMatrixDisplay, 116
DecisionTreeClassifier, 27
DummyClassifier, 32
GridSearchCV, 47, 80
GridSpec, 185
LabelEncoder, 34
LogisticRegression, 133
MeanMedianImputer, 10
OneHotEncoder, 10
PartialDependenceDisplay, 172
PartialDependencePlot, 176
Plot, 144
RandomForestClassifier, 52
RandomState, 23
RocCurveDisplay, 120
SeedSequence, 23
StandardScaler, 133
ThresholdXGBClassifier, 124
TreeExplainer, 152
Trial, 87
ValueError, 33
XGBClassifier, 61

XGBRFClassifier, 53
XGBlassifier subclass, 123
X, 9
accuracy_score, 115
add, 19
annotate, 125
annot, 162
ax.plot, 26
background_gradient, 15, 48
beeswarm, 165
calc_gini, 25
calibration, 185
choice, 88
classification_report, 120
cmap, 162
cm, 166
color, 166
colsample_bylevel, 54, 75
colsample_bynode, 54, 75
colsample_bytree, 54, 75
confusion_matrix, 115
corr, 162
create_experiment, 190
create_post_data, 197
cross_val_score, 48, 82
crosstab, 18, 182
cv_results_, 48
cv, 46, 48
depth_range_to_display, 54
dtreeviz, 30, 54
dummy, 32
early_stopping_rounds, 57, 67, 76, 85, 106
ensemble, 52
eval_metric, 57, 71, 76
eval_set, 67, 106, 132
export_graphviz, 138
exp, 154
extract_zip, 5
f1_score, 120
feature_engine, 10
fig.tight_layout, 176
fit_transform, 11, 31
fit, 31
fmin, 86, 105, 190
fmt, 162
force, 160
from_estimator, 172, 176, 185
gamma, 57, 76, 78
get_tpr_fpr, 124
groupby, 181
grow_policy, 54, 75
heatmap, 93
hp, 88
hyperopt, 105
initjs, 152
inspection, 172
interaction_constraints, 148
inv_log, 64
inverse_transform, 34
jitter, 94
json.loads, 195
lambda, 154
learning_curve, 109
learning_rate, 57, 76, 78, 79
linear_model, 133
load_model, 193
loads, 195
loc, 26
log_metric, 190
log_param, 190
loguniform, 89
max_delta_step, 57, 76
max_depth, 27, 40, 43, 54, 61, 75
max_evals, 105
max_features, 43
max_leaves, 43, 54, 75
min_child_weight, 54, 75
min_impurity_decrease, 43
min_samples_leaf, 43
min_samples_split, 43
min_weight_fraction_leaf, 43
mlflow ui, 192
model_selection, 11
model_selection (sklearn), 47
model, 135
monotone_constraints, 183
my_dot_export, 28, 54
n_estimators, 33, 52, 57, 61, 76, 132
np.exp, 154
num_trees, 62
objective, 57, 76
openpyxl, 141
orient='split', 195
param_range, 129
partial_dependence_plot, 176, 179
pd.crosstab, 18, 145, 182
pd.read_csv, 5
penalty=None, 133
plot.bar, 17
plot_3d_mesh, 100
plot_cumulative_gain, 125
plot_lift_curve, 126
plot_tree, 28, 31, 40, 52
plots.force, 160

plots.scatter, 160
plots.waterfall, 156
precision_recall_curve, 119
precision_score, 117
predict_proba, 185
preprocessing, 133
pyfunc, 193
pyll, 88
quantile_ice, 173
query, 10, 27, 182
quniform, 91
read_csv, 5
recall_score, 117
reg_alpha, 57, 76
reg_lambda, 57, 76
rounds, 105
run_id, 191
sample, 88
sampling_method, 54, 75
scale_pos_weight, 57, 76
scale, 21
scatter (SHAP), 160
score, 32, 45
scoring, 46, 129
set_experiment, 190
set_sticky, 15
shap (PDP), 179
so.Dots, 19, 144
so.Jitter, 144
so.Line, 19, 144
so.PolyFit, 144
spines, 146
stratify, 11
subsample, 54, 75
tight_layout, 176
to_json, 198
top_n, 10
tpe.suggest, 87
train_test_split, 11
transform, 11, 146
tree.plot_tree, 31, 52
tree_index, 62
tree_method, 54, 75
trial2df, 92
trial, 92
ui, 192
uniform, 89
urllib, 5
validation_curve, 46, 78, 129
value_counts, 182
verbose, 47, 81, 132
view, 135
vmax, 162
vmin, 162
x_jitter, 162
xg_helpers.py, 6
xg_helpers, 60
xgbfir, 141
y, 9
zipfile, 5
3D mesh, 106
3D plot, 100

accuracy, 32, 71, 115, 129
algorithm, 23
alpha, 162
annotate, 125
area under curve, 71, 121
arrow (on plot), 125
AUC, 71, 121
auc tuning, 130
Average Gain, 141
average precision, 119
Average wFScore, 141
axis spines, 146

bagging, 51
bar plot, 17, 137
base value, 156
baseline model, 32
bayesian optimization, 85
benefits of XGBoost, 59
binary feature, 165
black box, 60, 133, 151
boosting, 33
bootstrap aggregation, 51

calibration, 185
categorical, 88
categorical encoding, 31
categorical values, 59
centered ICE plots, 172
classification report, 120
clean up data, 31, 61
cleanup, 5
closure, 85
coefficient, 133
coefficients, 134
colormap, 166
column subsampling, 51
command line, 193
comparing models, 121
complicated, 40
Condorcet's Jury Theorem, 51

confusion matrix, 115
constraints, 148, 180
constructor, 43
conventions (scikit-learn), 52
convert labels to numbers, 34
correlation, 15, 93, 143
correlation between trees, 51
correlations, 162
cover, 137
cross tabulation, 145
cross validation score, 46
cross-validation, 82
cumulative gains curve, 125
curl, 193, 197

data leakage, 121
data size, 109
Decision stump, 27
decision tree visualization, 30
decrease learning rate, 79
default value, 156
dependence plot, 162
dependence plot (SHAP), 160
diagnose overfit, 111
diagnose underfit, 111
dictionary, 195
dictionary unpacking, 106
discrete, 88
distributions, 87
docker, 198
docker run, 198
DOT, 138
dtreeviz, 62, 135

early stopping, 67
EDA, 15
edge color, 23
endpoint, 193
ensemble, 51
ensembling, 54
environment, 193
error, 59
estimators, 52
eta, 79
evaluate labelling errors, 115
evaluation set, 67
Excel, 141
exhaust, 47
Expected Gain, 141
experiment tracking, 189
explaining, 60
explanations, 151
exploitation, 94
exploration, 94
exploratory data analysis, 15
export xgboost plot, 62

F1 score, 120
facet plot, 21
false positive, 25, 115, 117
feature engineering, 15
feature importance, 135
feature interactions, 141
fill in missing values, 31
first-class functions, 85
fit line, 19
fixing overfitting, 40
fixing underfit models, 39
font, 146
fraction (confusion matrix), 117
FScore, 141

gain, 125, 137, 141
gamma, 78, 100
Gini coefficient, 23, 25
global interpretation, 151
golfing, 59
good enough, 121
gradient descent, 33
graphviz, 138
grid search, 47, 80
grouping, 145
growing trees, 39

heahtmap, 162
heatmap, 93, 143
histogram, 23
histogram (ICE), 173
holdout, 11
holdout set, 11, 32
horizontal alignment, 146
Hyperopt, 85
hyperopt (with mlflow), 190
hyperparameter tuning, 87
hyperparameters, 39
hyperparameters (decision tree), 43
hyperparameters (random forest), 54
hyperparameters (XGBoost), 75

ICE, 171
ICE (shap), 176
imbalanced data, 11, 115
impute, 10
indicator column, 17

Individual conditional expectation, 171
integer distribution, 91
interaction, 141
interactions, 141
interactions (dependence plot), 162
interactive plot, 101
interactive visuals, 100
interpret, 133
interpretation, 60
inverse logit, 35, 64
isotonic calibration, 185

jitter, 19, 94, 162
JSON, 193
Jupyter, 197
juror, 51

k-fold validation, 82
Kaggle, 5

label encoder, 34
leaf values, 35
leaked data, 121
learning rate, 78
lift curve, 126
limit adjustment, 21
limit feature interactions, 148
line plot (Matplotlib), 26
load data, 5
local interpretation, 151
log loss, 71
log odds, 35, 154
log-axis, 100
log-uniform, 89
logistic regression, 133
logit, 35
loss, 100

macro average, 120
magnitude, 133
Matplotlib colormap, 166
max convention, 52
maximize marketing response rates, 125
memorizing, 40
metric, 46
min convention, 52
missing values, 59
mlflow, 189
mlflow serve, 193
monotonic constraints, 180

negative accuracy, 85
nested list, 148
no interactions (shap), 166
non-linear relationships, 135
non-linear response, 169
non-monotonic interactions, 165
normalize confusion matrix, 117
number of k-fold cross validations, 46
number of trees, 33
NumPy, 23

optimal gain, 125
optimal model, 126
optimize auc, 130
ordered samples, 125
over-fit model diagnosis, 109
overfit, 40
overfit diagnosis, 111
overfit learning curve, 111

Partial dependence plot, 176
PDP, 176
PDP (SHAP), 179
percentage (confusion matrix), 117
philosopher, 51
pipeline, 6, 9, 10, 31, 61
plot inverse logit, 36
plot polynomial line, 19
plot xgboost tree, 62
plotly, 100, 106
positive label, 34
precision, 117
precision-recall curve, 119
predict probability, 63
probabilities, 35
probability, 63, 124
probability of label, 119
prune, 40
pruning, 78
pyenv, 193
Python variables, 197

query model, 193

random forest, 51
random forest (XGBoost), 53
random forest hyperparameters, 54
random sample, 23
random samples, 23
rank, 141
ratio of gains, 126
recall, 117
receiver operating characteristic, 120
regularization, 54

requests, 198
residual, 59
results of evaluation, 68
reverse transformation, 34
ROC Curve, 120
roc-auc tuning, 130
roc_auc, 129
run name, 192
running a model, 193
running with docker, 198

sampling, 54
scale, 133
scatter plot, 19, 94, 144
Sci-kit learn pipeline, 6
scikit-learn conventions, 52
scikitplot, 125
Seaborn, 162
seaborn, 19, 93, 96, 144
sensitivity, 117
serve model, 193
service (mlflow), 192
setting y-axis limit, 109
shap, 151
SHAP (ICE), 176
shap waterfall, 156
shap without interactions, 166
sigmoid calibration, 185
sign of coefficient, 133
simple, 39
simpler models, 148
single prediction, 156
slope graph, 146
Spearman correlation, 15
specify feature interactions, 148
split, 11
split data, 61
split point, 23
standard deviation, 133
standardize, 133
step-wise tuning, 105, 130
sticky index, 15
string labels, 33
Stump, 27
stump, 166
subclass, 123
subsampling, 51
summary plot, 165
support, 120
survey data, 5

test set, 11
threshold, 120
threshold metrics, 124
tick location (Seaborn), 21
track artifacts, 192
trailing underscore convention, 52
trails, 87
training score, 46
training set, 11
transparency, 19
transparent model, 133
tree depth, 40
tree model, 23
trend, 19
true positive, 25, 115, 117
tuning function, 86
tuning hyperparameters, 78
tweak function, 6, 10

underfit, 39
underfit diagnosis, 111
undo transformation, 34
unstacked bar plot, 17
upack operatior, 47
uuid, 193

validation curve, 78
validation curve (random forest), 57
validation curves, 44
validation score, 46
validation set, 11, 32
variance, 40
verbose, 132
vertical alignment, 146
violin plot, 96
visualize decision tree, 31

waterfall plot, 156
weak model, 59
weight, 137
weighted average, 120
wFScore, 141
white box, 60, 133

x-axis log, 100
XGBoost hyperparameters, 75
XGBoost stump, 35

y-axis limit, 109
yellowbrick, 57, 120
Yellowbrick (validation curve), 46
yellowbrick learning curve, 109

ZIP file, 5